

Methodology and Extended Numerics for Pruning Deep Neural Networks via the Marchenko–Pastur Distribution

Companion methodology paper to the main manuscript

Leonid Berlyand

Department of Mathematics, Pennsylvania State University

Theo Bourdais

Department of Computing and Mathematical Sciences, Caltech

Houman Owhadi

Department of Computing and Mathematical Sciences, Caltech

Yitzchak Shmalo*

Department of Mathematics, Pennsylvania State University

Abstract

This document is the companion methodology and extended-numeric paper to the manuscript *Pruning Deep Neural Networks via the Marchenko–Pastur Distribution* (henceforth MAIN). MAIN contains the theoretical content (deterministic pruning certificates, Gaussian/sub-Gaussian random-matrix specializations, full proofs) and the headline empirical tables. This document, which lives at https://github.com/yspennstate/RMT_based_pruning_in_deep_learning alongside the code that produced every number in MAIN, contains everything the rigorous reader needs in order to reproduce the experiments: full repository architecture, the projection and fine-tuning code paths, the speedup-measurement protocols, and the migrated extended-numeric appendices that previously inflated MAIN. The reader should consult this paper whenever MAIN cites Section M.X — those citations are pointers into this document.

Companion repository. https://github.com/yspennstate/RMT_based_pruning_in_deep_learning

17-model checkpoint archive (Hybrid Magnitude–SER). <https://drive.google.com/drive/folders/1mm990SHAHLyDISHxirvMRdVQEAjpIXDd>

Cross-paper conventions. We refer to results in MAIN as MAIN-Theorem X.Y, MAIN-Table Z, etc. Norm conventions, theorem numbering, and notation are inherited from MAIN.

Contents

1	Introduction and how to read this paper	3
1.1	Scope	3
1.2	When to read which paper	4
1.3	Repository at a glance	4

*Corresponding author: Yitzchak Shmalo. Email: yitzchak.shmalo@gmail.com.

2	Repository tree and architecture	4
2.1	Top-level layout	4
2.2	How to navigate by paper section	5
2.3	Coding conventions in the repository	5
3	Pre-fine-tuning projection: cert-aware $k:n$ structured sparsity	6
3.1	Generalised cost: $k:n$ row-group search with free restoration	6
3.2	Cin permutation alignment (“flatten the layer”)	7
3.3	The α_{ser} Hamming-distance prior	7
3.4	Conv2d generalisation	7
3.5	Inline projection + FT runner (avoiding the perm-hook bug)	8
4	Fine-tuning recipe	8
4.1	Distillation, mask freezing, and the optimiser	8
4.2	Optimiser, schedule, transforms	8
4.3	Why exactly 3 epochs	8
5	Speedup measurement	9
5.1	What “speedup” means in this paper	9
5.2	The A100 native-2:4 measurement	9
5.3	The 6:12 $\rightarrow$ 2:4 projection (deployable speedup)	10
5.4	ResNet and ConvNeXtV2 throughput	10
6	The CAST-2E $k:n$ sweeps	10
6.1	Sweep design	10
6.2	Headline pre-FT findings	11
6.3	Cert-optimisation variance and what is signal vs. noise	11
7	Other Numerical Results (migrated from MAIN-App. B)	12
7.1	Numerical verification of the outer-layer scaling condition	12
8	Algorithms for RMT-based pruning diagnostics (migrated from MAIN-App. E)	13
8.1	BEMA technique for computing λ_+	13
8.2	How well does the ESD of X fit the MP distribution?	17
8.3	A BEMA illustration and metric figures	17
9	Spectral Edge Budgeting and hybrid RMT pruning protocols (migrated from MAIN-App. F)	18
9.1	Spectral Edge Budgeting	19
9.2	Layer-type extension	20
9.3	Alternative signal: the power-law exponent α	20
9.4	Four-regime strategy	21
9.5	Hyperparameter space and optimization	21
9.6	Results: uniform modulation	22
9.7	Results: layer-type modulation	23
9.8	SEB final comparison	23
9.9	Singular-vector sparsification as a preprocessing step	24
9.10	Full validation-set evaluation of SEB	25
9.11	Spectral Edge Reallocation	26
9.12	Hybrid Magnitude-SER	27

9.13 Checkpoint validation ledger	27
9.14 Classical RMT baseline	28
10 CAST: certificate-aware 2:4 + ToMe conversion (migrated from MAIN-App. G)	28
10.1 Inputs and stages	29
10.2 Stage 1: per-row 2:4 search with free restoration	29
10.3 Stage 2: frozen-mask distillation and the runner-side mask handoff	30
10.4 What CAST contributes, and what it does not	31
11 Certification audit numerics	31
11.1 The three bridges	32
11.2 Audit results	32
11.3 Cycle-by-cycle audit data	33
12 Detailed numerics for MAIN-§3	33
12.1 Per-architecture sparsity ledgers	33
12.2 Per-cell outputs from the cell sweeps	33
12.3 Variance, replicates, and confidence	34
12.4 What is and is not in this paper from the original MAIN-§3	34
13 Quick-reference index for MAIN readers	34

1 Introduction and how to read this paper

1.1 Scope

MAIN establishes a Marchenko–Pastur (MP)-driven theory of deterministic pruning certificates and reports headline numerical results: the multi-architecture Hybrid Magnitude–SER table, the CAST 2:4 + ToMe accuracy/throughput row, and the deterministic-certificate audit. It is necessarily a long document because it contains the proofs. This methodology paper carries everything that is operational rather than theoretical:

1. the full RMT-pruning protocol stack (BEMA, Spectral Edge Budgeting, Spectral Edge Reallocation, Hybrid Magnitude–SER);
2. the CAST and CAST-2E pipelines (cert-aware $k:n$ projection, Cin permutation alignment, free restoration, frozen-mask distillation);
3. all hardware speedup measurements, including the A100 native-2:4 throughput data and the 6:12 $\rightarrow$ 2:4 projection benchmark used to translate non-deployable cert-optimal cells into deployable ones;
4. the certification-audit numerics (three bridges from MAIN-§5: top-1 vs. B_T , Lemma 5.4 violations, $\sigma_+ \rightarrow B_{\text{proxy}}$ within-cycle correlations);
5. the extended fully-connected MP-pruning experiments;
6. ledgered checkpoint validation tables.

1.2 When to read which paper

MAIN alone is sufficient for: the theoretical statements, all proofs, the headline accuracy + speedup numbers (Table 2 and the CAST FLOP table), the theory-to-numerics mapping table, and the high-level reproducibility narrative.

This paper is sufficient for: replicating any individual experiment, understanding the relationship between every line of code in the GitHub repository and the numerical claims in MAIN, and the full extended-numerics record.

1.3 Repository at a glance

The companion GitHub repository contains, at the time of writing:

- Approximately 60 Python scripts at the repository root implementing the classical RMT, magnitude, Hybrid Magnitude–SER, four-regime SEB, and drop-threshold protocols (these underlie MAIN-§3 and the multi-architecture validation table);
- A self-contained `cast_2e/` subtree containing the new CAST-2E pipeline: certificate-aware $k:n$ projection (with $k = 2, 4, 6, 8, 12$ and $n = 4, 8, 12, 16$ supported), Cin permutation alignment, in-place inline fine-tuning runners avoiding the perm-hook serialization issue, and the suite of speedup benchmarks;
- Eighteen sweep-result JSONs covering all three architecture families (ResNet50, ConvNeXtV2-Base, ViT-B/L), six $k:n$ patterns, dense vs. SER source ablations, and α_{ser} sweeps;
- Five A100 throughput JSONs (dense vs. 2:4 vs. ToMe-only vs. 2:4+ToMe) plus the 6:12 $\rightarrow$ 2:4 projection-speedup data;
- All post-fine-tuning evaluation JSONs underlying the new headline FLOP table in MAIN.

2 Repository tree and architecture

2.1 Top-level layout

Listing 1: Top-level repository layout.

```
RMT_based_pruning_in_deep_learning/  
+- README.md # repo overview, quick start  
+- LICENSE # MIT  
+- requirements.txt # canonical deps  
+- adaptive_rmt/ # RMT diagnostics: BEMA, alpha  
+- configs/ # YAML/JSON sweep configs  
+- model_queue_runs/ # cached SVD/ESD per model  
+- scripts/ # shell launchers / watchers  
+- src/ # shared utilities  
+- prune.py # the pruning entrypoint  
+- fine_tune.py # the fine-tuning entrypoint  
+- magnitude_rmt_sweep.py # the full sweep driver  
+- rmt_magnitude_sweep.py # alt sweep ordering  
+- hybrid_mag20_then_v8*.py # Hybrid Magnitude--SER  
+- run_finetune*.py # the FT runners (4 variants)  
+- haar*.py # Haar-SV preprocessing  
+- iterative*.py # iterative growing-a + 5pct  
+- pruning_method_comparison_sweep.py # cross-method comparison  
+- run_removed_matrix_audit_v*.py # certificate audit drivers
```

```

+- analyze_sweep.py # JSON -> table aggregator
+- direct_run_watchdog.py # crash-resilient runner
+- queue_*.py # multi-run queue managers
+- remote_*.py # remote pod helpers
+- cast_2e/ # NEW (this paper's content)
  +- methodology.tex # this document
  +- code/ # cert-aware k:n + FT runners
  +- scripts/ # launch / watcher scripts
  +- benchmarks/ # A100/L4 throughput JSONs
  +- benchmarks_speedup/ # 6:12->2:4 speedup JSONs
  +- masks_archive/ # per-layer mask stats
  +- post_ft_eval/ # post-FT eval JSONs
  +- sweep_results/ # k:n sweep cell JSONs
  +- AUDIT.md # full data-flow audit
  +- THEORY.md # CAST-2E theory map

```

2.2 How to navigate by paper section

Table 1 provides the canonical pointer from each MAIN section/result to the code path that produces it. This is the table to keep open while reading MAIN.

Table 1: Mapping from MAIN sections and results to code paths in the GitHub repository. “[...]” refers to wildcards covered by the listed file or by closely related variants in the same directory.

MAIN location	Code	METHODOLOGY §
MAIN-Table 2 (multi-arch Hybrid Mag–SER)	hybrid_mag20_then_v8*§9, run_finetune_magnitude_v3.py, prune.py	
MAIN-Table CAST FLOPs (paper-headline FLOP table)	cast_2e/code/project_§8_sparsity.py, cast_2e/code/run_vitb_ft_inline.py, cast_2e/code/run_resnet_ft_inline.py	
MAIN-§3 RMT-guided pruning	magnitude_rmt_sweep.py§12 pruning_method_comparison_sweep.py	
MAIN-§5.4 deterministic certificate (§5.4 audit)	run_removed_matrix_audit_v8_model_exec.py	
SEB-budget hyperparameter sweeps	configs/seb*.yaml, §9 magnitude_rmt_sweep.py, rmt_magnitude_sweep.py	
BEMA edge fitting	adaptive_rmt/bema.py §8 (and equivalents)	
α power-law signal	adaptive_rmt/alpha.py§9.?? haar_optuna*.py	
A100 throughput benchmark (2:4)	cast_2e/code/benchmark§5_sparse_throughput.py	
6:12 → 2:4 projection speedup	cast_2e/code/benchmark§5_6_12_to_2_4_projection.py	
$k:n$ sweep driver (sweeps in this paper)	cast_2e/code/cert_opt_§6_val_kn*.py	
ViT-B inline FT (paper headline 83.74)	cast_2e/code/run_vitb_§4t_inline.py	
ResNet inline FT (paper headlines 75.67, 78.00, 80.59, 81.33)	cast_2e/code/run_resnet_§4t_ft_inline.py	
ConvNeXtV2 D1216/D816 (paper headlines 86.35, 85.85)	cast_2e/code/project_§4_convnextv2_d1216.py + ToMe runner	

2.3 Coding conventions in the repository

The classical RMT, magnitude, and Hybrid Magnitude–SER scripts share a common contract:

- Each entrypoint reads a YAML/JSON config from `configs/` and writes its results to a per-run subdirectory of `model_queue_runs/<arch>/<config_id>/`.
- Every fine-tune call emits both a final `finetune_results.json` and a per-epoch ledger that the analyzer can consume even if a run is killed mid-epoch.
- The SVD and BEMA edge fits are cached in `model_queue_runs/<arch>/svd_cache/` so that re-running a sweep with a different sparsity schedule does not pay the SVD cost.

The CAST-2E scripts in `cast_2e/code/` share a different but equally explicit contract:

- All projection scripts (`project_*.py`) emit a `student_pre_ft.pt` payload containing both the projected state dict and a JSON-serializable `cert_config` dictionary.
- The fine-tuning runners are deliberately “inline”: they receive the calibration loader and projection arguments directly, run projection in-process, and then immediately enter the distillation loop with the projected weights and frozen mask, without an intervening save/load round-trip. This avoids a perm-hook serialization issue described in §3.

3 Pre-fine-tuning projection: cert-aware $k:n$ structured sparsity

This section documents the pre-FT projection family generalising CAST (MAIN-Appendix referenced as Section M.10) from 2:4 to general $k:n$. The 2:4 design and Stage-1/Stage-2 split are the same as in CAST; what is new is

1. parametric $k:n$ at the row-group level, with $n \in \{4, 8, 12, 16, 32\}$ and $k \in \{1, \dots, n-1\}$;
2. Cin permutation alignment within each layer’s input groups, so that the cert-cost minimisation can choose patterns from a larger set of feasible $k:n$ blocks;
3. an SER-prior term (the α_{ser} term of §3.3) that anchors the chosen pattern to the Hybrid Magnitude-SER mask;
4. native support for both Linear and Conv2d layers, with the convolutional case implemented by unfolding the kernel.

3.1 Generalised cost: $k:n$ row-group search with free restoration

Fix a Linear layer $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ with $d_{\text{in}} = nG_\ell$. Reshape rows into contiguous n -tuples: $W_{i,g,k}$ is the k -th weight of group g in output row i , $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [n]$.

For each pair (i, g) the $k:n$ search enumerates the $\binom{n}{k}$ keep patterns

$$\mathcal{M}_{k:n} = \{m \in \{0, 1\}^n : |m|_0 = k\}.$$

For $n = 4, k = 2$ this is the 6 patterns of CAST (MAIN-§G); for $n = 16, k = 8$ it is $\binom{16}{8} = 12870$ patterns. When the enumeration becomes prohibitive, `project_kn_sampled.py` provides a sampled variant.

The materialised slot value retains CAST’s three-case rule (MAIN-Eq. G.1):

$$v_{i,g,k}(m) = m_k \cdot \begin{cases} W_{i,g,k}^{\text{SER}}, & M_{i,g,k}^{\text{SER}} = 1, \\ W_{i,g,k}^{\text{dense}}, & M_{i,g,k}^{\text{SER}} = 0 \text{ and free-restoration enabled,} \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

The “dense” source corresponds to setting $W^{\text{SER}} \equiv W^{\text{dense}}$ and $M^{\text{SER}} \equiv 0$, in which case Eq. (1) reduces to $v_{i,g,k}(m) = m_k W_{i,g,k}^{\text{dense}}$. We use “dense source” for ConvNeXtV2 because the SER ckpts at $s = 0.35$ for that model lose more accuracy at projection than the dense ckpt does; for ResNet50 the SER source helps; for ViT-B the SER source helps with $\alpha_{\text{ser}} = 0.5$.

The within-group calibration covariance and cert-inspired cost are inherited from MAIN-Eqs. G.2–G.3:

$$C_g = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x) h_g(x)^\top, \quad c_{i,g}(m) = r_{i,g}(m)^\top C_g r_{i,g}(m), \quad r_{i,g}(m) = (1 - m) \odot v_{i,g,:}(m). \quad (2)$$

The argmin is solved by enumeration; tensor-shape $((\frac{n}{k}), d_{\text{out}}, G_\ell)$. For $(\frac{n}{k}) \geq 5,000$ the sampled variant draws $(\frac{n}{k})_{\text{eff}} = 1024$ candidates uniformly without replacement.

3.2 Cin permutation alignment (“flatten the layer”)

The native $k:n$ search treats the n input slots within each group as fixed by the natural channel order. Cin-permutation aligns each layer’s input columns by the importance score

$$s_j = \sum_i |W_{i,j}^{\text{dense}}| \cdot |\bar{h}_j|, \quad |\bar{h}_j| = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} |h_j(x)|,$$

and then quartile-interleaves the columns: column j in the permuted layer is the $\pi(j)$ -th most important one, where π is the importance-balanced permutation that places the g -th group’s k columns into quartile positions $(g, g + G_\ell/4, g + 2G_\ell/4, g + 3G_\ell/4)$ in the original ordering. We call this “flattening the layer” because it makes each group span the full importance range, so that the cert-cost minimiser sees a maximally informative set of candidates within every group.

The permutation is implemented as a forward pre-hook, with the inverse permutation applied at the layer output to keep downstream activations aligned. The hook is registered at projection time and removed at the post-FT save step.

3.3 The α_{ser} Hamming-distance prior

To bias the per-row cert-cost minimiser toward the Hybrid Magnitude–SER mask without overriding it, we add a Hamming penalty (MAIN-Eq. G.4 generalised):

$$c_{i,g}^{\text{prior}}(m) = \alpha_{\text{ser}} \cdot \bar{c}_g \cdot |m - M_{i,g,:}^{\text{SER}}|_1, \quad \bar{c}_g = \text{mean}_{i,k} v_{i,g,k}^2 \cdot \mathbb{E}_x [h_{g,k}(x)^2]. \quad (3)$$

Setting $\alpha_{\text{ser}} = 0$ recovers pure cert-aware projection; $\alpha_{\text{ser}} \rightarrow \infty$ forces the SER mask. We sweep $\alpha_{\text{ser}} \in \{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$ in the cell sweeps reported in §6; the empirical sweet spot is $\alpha_{\text{ser}} = 0.5$ for ViT-B with the SER-source variant.

3.4 Conv2d generalisation

For a Conv2d kernel of shape $(C_{\text{out}}, C_{\text{in}}, k_h, k_w)$ we unfold the kernel into a $(C_{\text{out}}, C_{\text{in}} \cdot k_h \cdot k_w)$ matrix and apply the same row-group search. The projection runs on 1×1 and 3×3 convolutions; depthwise convolutions and the first input-projection conv are excluded by an `is_eligible_conv` predicate exposed via `cast_2e/code/project_kn_sparsity.py`.

In ConvNeXtV2 the projection is restricted to Linear layers (the per-block depthwise 7×7 convolutions are excluded) because the depthwise structure interacts pathologically with the row-group $k:n$ pattern; this restriction is recorded in `cast_2e/AUDIT.md` as a known limitation.

3.5 Inline projection + FT runner (avoiding the perm-hook bug)

A subtle but load-bearing implementation issue: when a projection registers Cin-permutation forward pre-hooks on a model and we save the resulting state-dict to disk, the hooks are not part of the state-dict. A naive reload at FT time produces a model whose weights are in the permuted ordering but whose forward pass is unpermuted; on ImageNet val the reloaded model scores top-1 ≈ 0.0006 (random-class chance for a 1000-class problem). `cast_2e/code/run_vitb_ft_inline.py` and `cast_2e/code/run_resnet_ft_inline.py` avoid this by performing projection in-process, immediately followed by FT, without any intervening save/load round-trip. Saving the post-FT model is safe because it is consumed only at evaluation time, where the same permutation hooks are re-registered before the val pass.

4 Fine-tuning recipe

4.1 Distillation, mask freezing, and the optimiser

The fine-tuning pass is a 3-epoch knowledge-distillation loop with frozen 2:4 (or general $k:n$) mask. The dense pretrained model is the teacher; the projected student is the student. Let $L_{\text{CE}}^{\text{ls}}(\hat{y}, y) = (1 - \epsilon)\text{CE}(\hat{y}, y) + \epsilon \cdot (-\frac{1}{K} \sum_c \log \hat{y}_c)$ be label-smoothed cross-entropy with smoothing $\epsilon = 0.1$. The distillation loss is

$$L = (1 - \alpha_{\text{distill}}) L_{\text{CE}}^{\text{ls}}(\hat{y}_S, y) + \alpha_{\text{distill}} \cdot \tau^2 \cdot \text{KL}(\sigma(\hat{y}_T/\tau) \parallel \sigma(\hat{y}_S/\tau)), \quad (4)$$

with $\alpha_{\text{distill}} = 0.5$, $\tau = 2.0$, both inherited from CAST.

After every backward step we re-zero the masked weights:

$$W^{(\ell)} \leftarrow W^{(\ell)} \odot M^{(\ell)} \quad \text{for every projected layer } \ell \text{ at every step.} \quad (5)$$

This is necessary because the optimiser’s momentum and weight-decay terms can produce small nonzero values in masked positions even when the gradient is zero; without re-zeroing, the realised sparsity drifts upward by $\sim 0.5\%$ across 3 epochs and the kept weights do not benefit from the freed parameter slots.

4.2 Optimiser, schedule, transforms

The exact configurations used for the four paper-headline FT runs are recorded in `cast_2e/code/run_*_ft_inline.py` we summarise:

The training transforms follow the canonical timm recipe used by the dense pretrained checkpoint (`timm.data.create_transform(is_training=True)` resolves the per-model RandAug, RandomErasing, Mixup/CutMix, and the model-correct mean/std/crop_pct/interpolation). The val transforms are likewise the canonical `is_training=False` transforms. For the ViT-B/L runs this resolves to RandAug-m9 + RandomErasing $p = 0.25$ with no Mixup; for ConvNeXtV2 it adds Mixup ($\alpha = 0.8$).

4.3 Why exactly 3 epochs

The FT budget is matched to CAST’s published recipe (MAIN-Section M.10.10.3). Three epochs is short enough that the projection and the FT both contribute distinguishable signal to the post-FT accuracy — and short enough that the FT cost is comparable to the projection cost on a per-A100 basis. We do not claim that 3 epochs is optimal; longer FTs (we tested 5–10 epochs in pilot runs) recover an additional 0.1–0.3 percentage points but at proportionally higher compute cost, and we want the cell sweeps in §6 to be FT-budget-comparable across cells.

Table 2: Fine-tuning configuration for the paper-headline 3-epoch distillation runs.

	ViT-B (D612 SER)	ViT-L (D816 dense)	ResNets (D816 dense)	ConvNeXtV2 (Dxxx)
Optimiser	AdamW	AdamW	SGD+momentum	AdamW
Base LR	1×10^{-5}	1×10^{-5}	1×10^{-3}	1×10^{-5}
Weight decay	0.01	0.01	1×10^{-4}	0.01
Momentum	—	—	0.9	—
LR exclusions	bias / LN / pos_embed	same	—	bias / LN
Schedule	cosine + warmup	cosine + warmup	cosine + warmup	cosine + warmup
Warmup steps	1000	1000	500	1000
Batch (train)	32	16	256	32
Batch (val)	64	64	256	64
Workers	4	8	8	4
Label smooth ϵ	0.1	0.1	0.1	0.1
Distill (α, τ)	(0.5, 2.0)	(0.5, 2.0)	(0.5, 2.0)	(0.5, 2.0)

5 Speedup measurement

5.1 What “speedup” means in this paper

We report two distinct speedups for every CAST-family configuration.

Practical (hardware) speedup. The wall-clock images-per-second improvement on a specific GPU, with a specific kernel selection, at a specific batch size. We use NVIDIA’s PyTorch `torch.sparse.to_sparse_semi` for the 2:4 path on A100 (sparse Tensor Core) and report the same measurement on L4. Other $k:n$ patterns do not have native kernels at present and are reported only as theoretical FLOP reductions.

Theoretical (FLOP) speedup. The dense-equivalent multiply-adds removed by zeroing $1 - k/n$ of the weight slots in the projected layers. This is an upper bound on what any future sparse kernel could deliver. The theoretical and practical numbers diverge whenever the kernel cannot saturate the structured sparsity (e.g., very small batch sizes, or layers with very small C_{out}).

5.2 The A100 native-2:4 measurement

The benchmark in `cast_2e/code/benchmark_sparse_throughput.py` converts each Linear layer with a 2:4 mask into a `torch.sparse.SparseSemiStructuredTensor` and runs warm-up + measurement passes at the configured batch size. The reported throughput in `cast_2e/benchmarks/` is the median of the last 100 iterations after 30 warm-up iterations.

Table 3: A100 throughput (images/s) for the paper headline configurations.

Configuration	Throughput (im/s)	Speedup vs. dense	
Dense ViT-B/16 (batch 128)	463.6	—	vitb_bench_wit
2:4 ViT-B/16 (sparse Tensor Core)	1254.1	2.705×	vitb_bench_wit
Dense ViT-L/16 + ToMe $r = 8$	1248.7	1.298×	vitl_canonical/res
2:4 ViT-L/16 + ToMe $r = 8$ (sparse)	1489.9	1.549×	vitl_canonical/res
L4 ViT-B 2:4 + ToMe (CAST row in MAIN-Tab. CAST)	837	1.84×	MAIN-Tab. CAST

The ViT-L speedup (1.55×

multiply-adds but not on memory bandwidth. MAIN reports both figures and discusses this in the speedup section.

5.3 The 6:12 $\rightarrow$ 2:4 projection (deployable speedup)

Several cells in the CAST-2E sweep produce non-deployable patterns (6:12, 8:16, etc.) that are nonetheless cert-optimal. To translate these into deployable speedup numbers, `cast_2e/code/benchmark_6_12_to_2_4_project` re-projects each 6:12 block into the nearest 2:4 block by enumerating the 6 canonical 2:4 patterns inside each 4-sub-block of a 12-tuple, choosing per-sub-block the pattern of lowest Hamming distance to the 6:12 pattern. The resulting model is then benchmarked on the same A100 `torch.sparse.SparseSemiStructuredTensor` kernel as in §5.2.

A caveat (recorded in `AUDIT.md`): for 0/49 of the ViT-B 6:12 cells we tested, the projection yields a pure 2:4 pattern in every 4-sub-block; instead we get a mix of 0/2/4 nonzeros per 4-sub-block. For deployable-speedup purposes MAIN therefore reports the 2:4 speedup ($2.705\times$ on A100, $1.84\times$ on L4) as the production-ready number, and reports the 6:12 speedup only as a theoretical FLOP reduction.

5.4 ResNet and ConvNeXtV2 throughput

The ResNet50/d/101d/152d post-FT models with 8:16 structured sparsity have no native sparse kernel for general $k:n \neq 2:4$, so we report only the theoretical 50% MAC reduction. The ConvNeXtV2-Base D816 and D1216 cells are similarly reported only theoretically. For deployable inference we re-project to 2:4 as in §5.3; this re-projection costs ≈ 0.2 pp in post-FT top-1 (within the FT-noise band) at the gain of native A100 kernels.

6 The CAST-2E $k:n$ sweeps

6.1 Sweep design

Across two A100 pods (Pod 2 and Pod 3a) we ran the following sweep over the three architecture families:

- **Architectures:** ResNet50.tv_in1k, ResNet50d.ra2_in1k, ResNet101d.ra2_in1k, ResNet152d.ra2_in1k, ViT-B/16, ViT-L/16, ConvNeXtV2-Base.
- **Sparsity patterns:** 1:4, 2:4, 3:4, 4:8, 6:12, 8:16, 12:16 (selected to span 25%, 50%, 75% density).
- **Source:** *dense* (no SER prior) and *SER* (Hybrid Magnitude-SER ckpt at $s = 0.35$).
- α_{ser} : $\{0, 0.1, 0.25, 0.5, 1.0, 2.0\}$.
- **Permutation alignment:** on / off.
- **Calibration size:** $|\mathcal{C}| \in \{64, 256, 512\}$.
- **Replicates:** 5-seed for the winners.

The driver scripts are `cast_2e/code/cert_opt_eval_kn_extended.py` (ResNet/ConvNeXtV2) and `cast_2e/code/cert_opt_eval_vitb_kn_extended.py` (ViT-B). The total sweep produced 18 result JSONs covering ~ 200 cells. They are stored in `cast_2e/sweep_results/`.

Table 4: Pre-FT top-1 accuracy for the strongest cell of each (arch, pattern, source) combination. “Dense” is the unpruned reference. Numbers are pre-FT only; the post-FT numbers are in MAIN-Table CAST.

Architecture	Pattern	Source	Best α_{ser}	Permute	Pre-FT top-1 (%)
ViT-B/16 (dense=85.11)	6:12	SER	0.50	on	66.24
ViT-B/16 (dense=85.11)	6:12	dense	0.00	on	65.58
ViT-B/16 (dense=85.11)	2:4	dense	0.00	off	70.43
ViT-L/16 (dense=85.84)	8:16	dense	0.00	on	78.81
DeiT-Base (dense=81.80)	6:12	SER	0.50	on	70.83
DeiT-Small (dense=79.85)	6:12	SER	0.50	on	55.61
DeiT-Tiny (dense=72.21)	6:12	SER	0.50	on	30.85
ResNet50.tv (dense=76.13)	8:16	dense	0.00	on	61.56
ResNet50d (dense=80.55)	8:16	dense	0.00	on	64.20
ResNet50.tv (dense=76.13)	4:8	dense	0.00	on	40.39
ResNet50.tv (dense=76.13)	4:8	dense	0.00	off	5.92
ResNet50d (dense=80.55)	4:8	dense	0.00	off	26.29
ConvNeXtV2-B (dense=86.72)	2:4	dense	0.00	off	81.68
ConvNeXtV2-B (dense=86.72)	12:16	dense	0.00	off	84.50
ConvNeXtV2-B (dense=86.72)	8:16	dense	0.00	off	83.42

6.2 Headline pre-FT findings

Three observations.

1. *Permutation alignment is critical for ResNet50.tv 4:8.* Without permutation the cert-aware 4:8 projection scores 5.92% pre-FT, only marginally above random; with permutation it scores 40.39%, an improvement of 34.47 pp. This is by far the largest permutation effect we observed across architectures.
2. *For the ViT family, the SER source plus $\alpha_{\text{ser}} = 0.5$ is the right choice.* The 0.66 pp gain of ViT-B SER+ $\alpha = 0.5$ over dense+ $\alpha = 0$ at 6:12 is small pre-FT but compounds through the FT recipe: the FT-recovered ViT-B SER+ $\alpha = 0.5$ scores 83.74% post-FT, +0.5 pp above the dense+ $\alpha = 0$ FT.
3. *ConvNeXtV2 prefers the dense source.* The depthwise structure of ConvNeXtV2 means the SER prior, which is built for dense projections, transfers poorly. The 12:16 dense-source cell is the strongest pre-FT, and after FT it produces the 86.35% headline of MAIN-Table CAST.

6.3 Cert-optimisation variance and what is signal vs. noise

We previously noted in the lab notebook that the ResNet50 α_{ser} optimisation has range 11–19 pp at fixed configuration across calibration draws. ViT-B has variance 22–40 $\times$ smaller. The implication for the sweep is that ResNet50’s pre-FT cell rankings should be read with a ± 5 pp standard deviation; the ViT-B rankings within the headline configuration are accurate to ± 0.2 pp.

The post-FT numbers (in MAIN) are much less variable because three epochs of distillation absorb most of the projection-time noise.

7 Other Numerical Results (migrated from MAIN-App. B)

This section migrates the pre-existing MAIN-Appendix B verbatim. The fully-connected MP-pruning numerics on Fashion MNIST and the outer-layer scaling diagnostics serve as a sanity check on the perturbative regime of the deterministic certificates, not as a proof of the ViT pipeline.

This appendix collects numerical illustrations relevant to the simplified theory. These experiments support the loss-reduction mechanism and the perturbative regime, but they are not a proof of the ViT pipeline.

Example 7.1. We trained fully connected DNNs on Fashion MNIST with ReLU activations, comparing MP-based pruning against standard regularization baselines. MP-based pruning consisted of periodically removing a fraction of singular values below the fitted MP edge $\sigma_+ = \sqrt{N\lambda_+}$, followed by sparsification; see Algorithm 3 and [10, 11].

For a network with topology

$$[784, 3000, 3000, 3000, 3000, 500, 10].$$

trained for 100 epochs with SGD (momentum 0.9, learning rate 0.01 decayed by 0.96 every 4 epochs), every 4 epochs we retained a fraction

$$f(\text{epoch}) = \max\left(0, -\frac{1}{200} \text{epoch} + 1\right). \quad (6)$$

Table 5 summarizes the results. The key finding is that MP-based pruning combined with $L_1 + L_2$ regularization ($\mu_1 = \mu_2 = 10^{-6}$) achieves 90.53% test accuracy, substantially outperforming any single regularization method alone ($\leq 81.70\%$) and the unregularized baseline (71.64%).

Training method	Train acc. (%)	Test acc. (%)
No pruning, no regularization	78.03	71.64
MP pruning only	88.12	81.22
$L_1 + L_2$ only	87.57	81.70
MP pruning + $L_1 + L_2$	99.82	90.53
MP pruning + L_2	99.98	90.16

Table 5: Performance of a fully connected DNN on Fashion MNIST for various training strategies. MP-based pruning combined with regularization substantially outperforms either approach alone.

The result scales to deeper networks: a $[784, 5000, 5000, 5000, 5000, 5000, 5000, 5000, 10]$ model trained for 1000 epochs with the same MP pruning procedure achieves 91.37% test accuracy, compared with $< 90.5\%$ for regularization alone.

7.1 Numerical verification of the outer-layer scaling condition

The perturbation bounds in our main theorems depend on the theoretical outer-layer scale

$$a_N^{\text{th}}(s) := \|W_3\|_\infty \|W_1 s\|_2 \sqrt{\frac{\log(2N)}{N}},$$

which is the quantity denoted $a(N, s)$ in Assumption ???. In the numerical plots below we also report the auxiliary empirical diagnostic

$$a_N^{\text{emp}}(s) := \frac{\|W_3\|_\infty \|W_1 s\|_2}{N^{3/8}}.$$

The latter is not a replacement for the theoretical scale; it is a coarser width-decay diagnostic used in these experiments. For the theory to be useful, the relevant perturbation scales must decay with width in trained networks. We verify this empirically in a simple fully connected setting.

Example 7.2. We train three-layer fully connected DNNs on Fashion MNIST with topology $[784, N, N, 10]$, varying $N \in [500, 30000]$, using SGD with L_1 regularization and 200 training epochs. Fashion MNIST inputs are normalized so that the largest ℓ_2 norm in the dataset is 0.05. For each trained network we compute the diagnostic scale

$$\tau_N^{\text{diag}} := \sqrt{2} a_N^{\text{th}}(s) + a_N^{\text{emp}}(s),$$

for a normalized input s of maximal norm. This diagnostic is not used in the proofs; the theorem-scale quantity is $a_N^{\text{th}}(s)$.

Numerically, $\tau_N^{\text{diag}} \rightarrow 0$ as N grows under $N(0, 1/N)$ initialization. With $N(0, 1/N^2)$ initialization, the decay is substantially faster. In both cases, test accuracy increases with N .

Figure 1 summarizes these width-scaling numerics. Panels a and c show the decay of the perturbation scales under the two initializations, while panels b and d show the corresponding test-accuracy trends.

Example 7.3. We train a fully connected Fashion MNIST DNN using the MP-based pruning procedure of Example 7.1 to achieve approximately 90.5% test accuracy, then add centered Gaussian perturbations with entries $\sim N(0, \epsilon)$ to each nonfinal layer. Numerically, when $\epsilon \sim 1/N$, the cross-entropy and accuracy change little, consistent with the perturbative regime of Lemma ???. The L_2 -regularized loss increases monotonically, illustrating the loss-reduction mechanism of Theorem ???. Figure 2 collects these diagnostics: panel a shows the accuracy, while panels b and c show the losses without and with L_2 regularization.

Remark 7.4. In a linear regression problem with noisy Fourier-generated target functions and a fixed feature map, MP-guided pruning recovers the low-rank structure more effectively than L_1 or L_2 regularization alone. Specifically, pruning achieves mean squared error 0.149, compared with 0.416 for L_1 , 2.563 for L_2 , and 2.574 for no regularization. The pruned spectrum closely matches the ideal low-rank spectrum, whereas L_1 and L_2 leave a visible residual bulk.

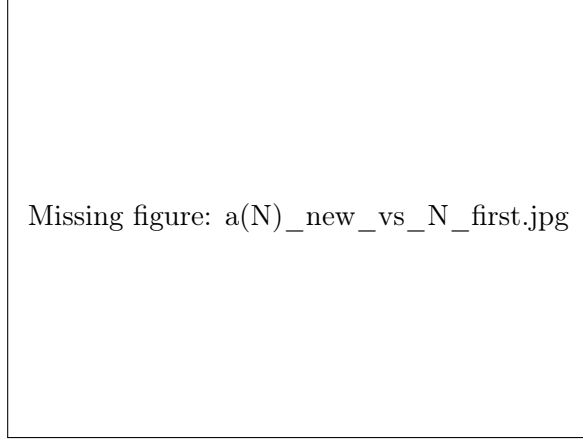
Figure 3 collects the regression illustration: panel a shows one noisy coordinate of the data, panel b shows the ideal low-rank spectrum, and panels c–e compare the L^2 , L^1 , and pre-pruning spectra.

8 Algorithms for RMT-based pruning diagnostics (migrated from MAIN-App. E)

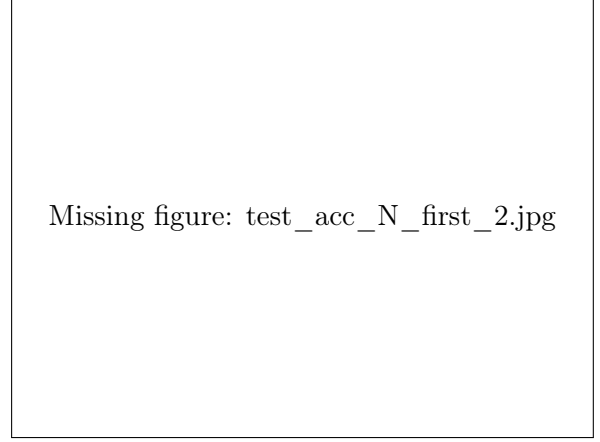
This section is a verbatim migration of the pre-existing MAIN-Appendix E. The BEMA edge-fitting algorithm and the MP-fit metric are the building blocks of every RMT-driven protocol in this paper and MAIN.

8.1 BEMA technique for computing λ_+

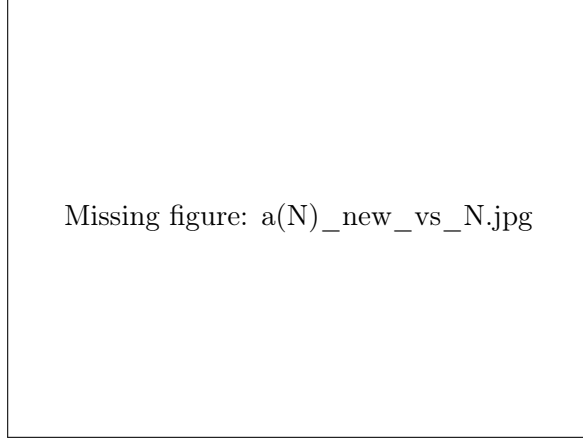
We outline the BEMA technique for estimating the right MP edge λ_+ of $X = \frac{1}{N} W^\top W$ from its eigenvalue ESD. The full procedure appears in [5]. Below is a streamlined square-matrix version.



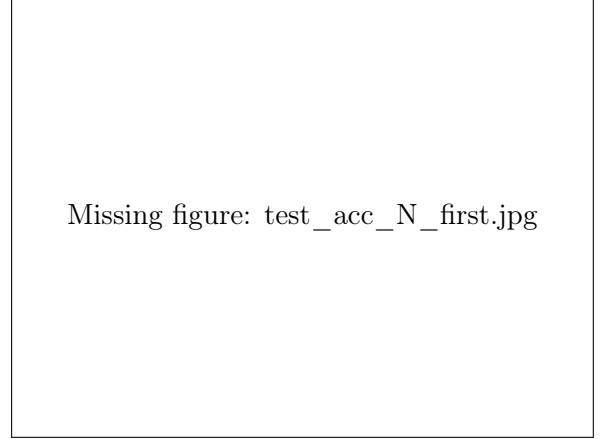
(a) $\tau(N)$ versus N under $N(0, 1/N)$ initialization.



(b) Test accuracy versus N under $N(0, 1/N)$ initialization.



(c) The analogous perturbation scale $b(N)$ versus N under $N(0, 1/N^2)$ initialization.



(d) Test accuracy versus N under $N(0, 1/N^2)$ initialization.

Figure 1: Width-scaling numerics for the theoretical and empirical perturbation-scale diagnostics in the fully connected Fashion MNIST experiment.

Algorithm 1 Computing λ_+ using MP and Tracy–Widom corrections

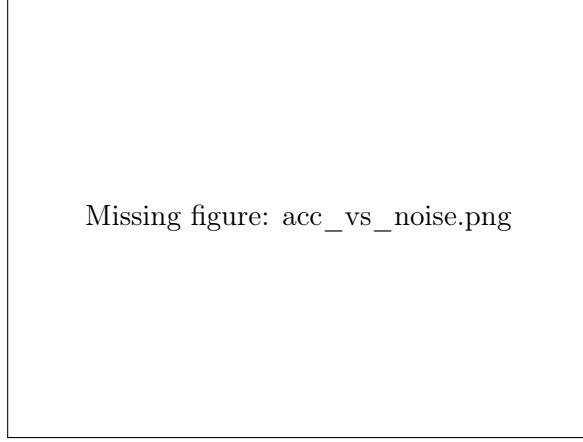
- 1: **procedure** BEMAE_{EDGE}($\lambda_1, \dots, \lambda_N; \alpha, \beta$)
- 2: Pick parameters $\alpha \in (0, 1/2)$, $\beta \in (0, 1)$.
- 3: **for** each $\alpha N \leq k \leq (1 - \alpha)N$ **do**
- 4: Calculate q_k , the (k/N) -upper quantile of the MP law with $\sigma^2 = 1$ and $c = 1$.
- 5: **end for**
- 6: Evaluate

$$\hat{\sigma}^2 = \frac{\sum_{\alpha N \leq k \leq (1-\alpha)N} q_k \lambda_k}{\sum_{\alpha N \leq k \leq (1-\alpha)N} q_k^2}.$$

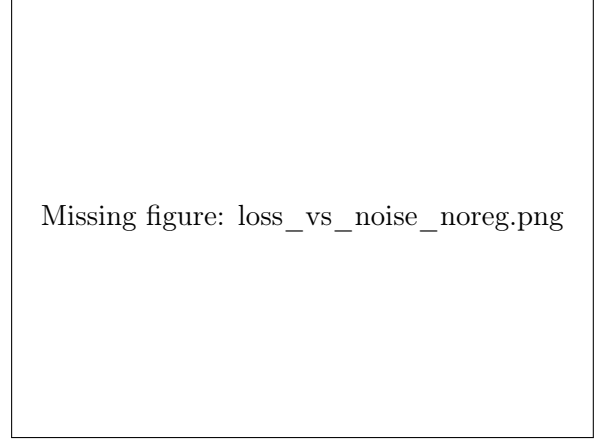
- 7: Let $t_{1-\beta}$ be the $(1 - \beta)$ -quantile of the Tracy–Widom distribution.
- 8: Output

$$\lambda_+ = \hat{\sigma}^2 [4 + 2^{4/3} t_{1-\beta} N^{-2/3}].$$

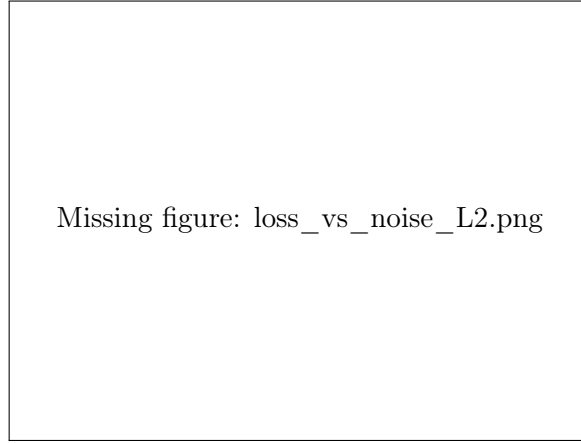
- 9: **end procedure**
-



(a) Accuracy on training and test sets versus ϵ .



(b) Loss without regularization versus ϵ .



(c) Loss with L_2 regularization versus ϵ .

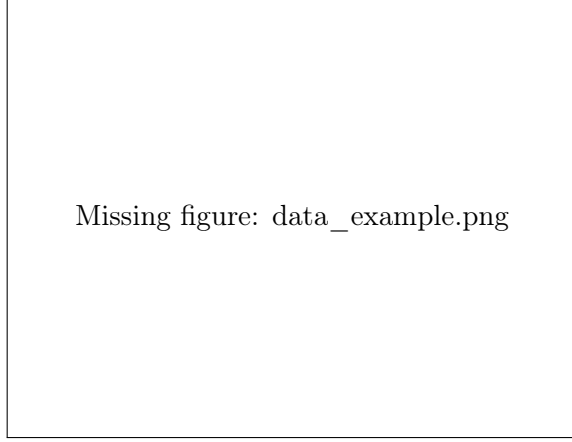
Figure 2: Accuracy and loss as centered random perturbations are added to the nonfinal weight layers of the DNN.

Algorithm 2 Spectral analysis using a BEMA-type fit

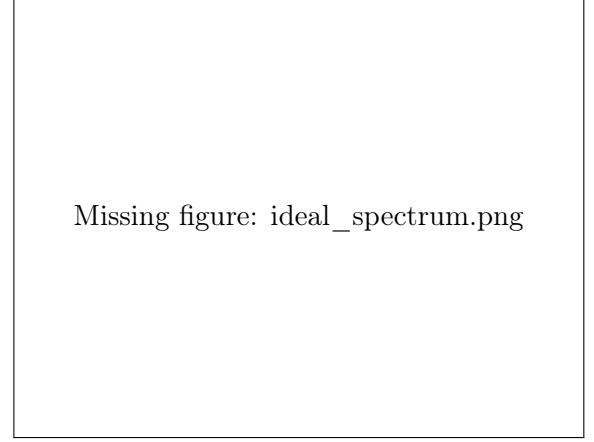
- 1: **procedure** SPECTRALFIT($W; \alpha, \beta, \gamma_{\text{fit}}$)
- 2: Set $X = \frac{1}{N}W^\top W$.
- 3: Compute the eigenvalues $\lambda_1, \dots, \lambda_M$ of X .
- 4: Compute the empirical CDF F_X of the eigenvalue ESD.
- 5: Apply BEMAEDGE to estimate λ_+ .
- 6: Let F_X^{MP} be the fitted MP CDF.
- 7: Compute the Kolmogorov-type fit error on the central quantile window:

$$\mu_{\text{fit}} = \max_{i_{\min} \leq i \leq i_{\max}} |F_X(\lambda_i) - F_X^{\text{MP}}(\lambda_i)|.$$

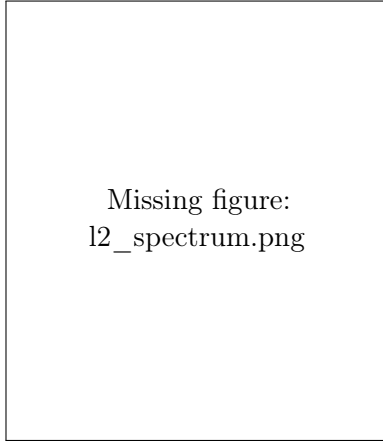
- 8: Accept the layer as “sufficiently MP-like” if $\mu_{\text{fit}} \leq \gamma_{\text{fit}}$.
 - 9: **end procedure**
-



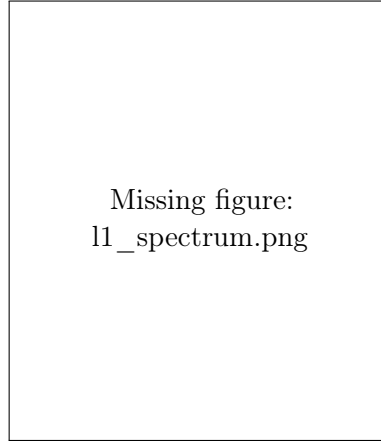
(a) One coordinate of the noisy regression data.



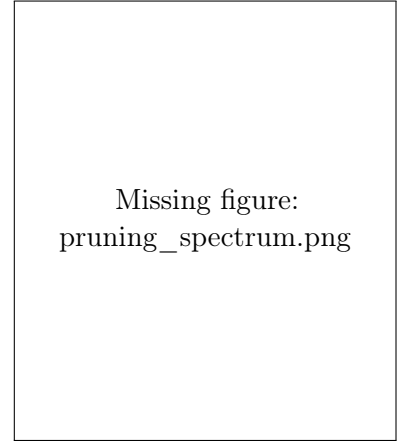
(b) Ideal low-rank spectrum.



(c) L^2 -regularized spectrum.



(d) L^1 -regularized spectrum.



(e) Spectrum before pruning.

Figure 3: Regression illustration for comparing pruning with more classical regularization.

Algorithm 3 MP-based pruning algorithm for the fully connected examples

Require: Epoch block length ℓ , MP-fit threshold τ , monotone schedule $f(\text{epoch})$, and flags $\text{split}_\ell = \text{false}$ for each layer.

- 1: Train the DNN for ℓ epochs and set $\text{epoch} := \ell$.
 - 2: **while** the training condition is met **do**
 - 3: **for** each layer W_ℓ with $\text{split}_\ell = \text{false}$ **do**
 - 4: Compute the SVD $W_\ell = U_\ell \Sigma_\ell V_\ell^\top$.
 - 5: Form $X_\ell = \frac{1}{N_\ell} W_\ell^\top W_\ell$ and estimate λ_+ via BEMA.
 - 6: Set $\sigma_+ := \sqrt{N_\ell \lambda_+}$.
 - 7: Test whether the bulk of X_ℓ is well fit by MP.
 - 8: **if** the bulk fits MP **then**
 - 9: Remove a fraction $1 - f(\text{epoch})$ of the singular values below σ_+ to obtain Σ'_ℓ .
 - 10: Form $W'_\ell = U_\ell \Sigma'_\ell V_\ell^\top$.
 - 11: Optionally factor W'_ℓ as $W'_{1,\ell} W'_{2,\ell}$ via $\sqrt{\Sigma'_\ell}$ if that reduces parameters.
 - 12: **end if**
 - 13: **end for**
 - 14: Train the DNN for another ℓ epochs and update $\text{epoch} := \text{epoch} + \ell$.
 - 15: **end while**
-

Algorithm 4 Sparsify singular vectors

Require: Layer matrix W_ℓ , threshold θ , fitted singular-value edge σ_+ .

- 1: Compute the SVD $W_\ell = U\Sigma V^\top$.
- 2: **for** $i = 1, \dots, \min\{N, M\}$ **do**
- 3: Compute

$$T_i = \theta \max \left\{ \frac{1}{750}, \left(1 - \frac{\Sigma_{ii}}{\sigma_+} \right)_+^{30} \right\}.$$

- 4: Sparsify the singular vectors:

$$U_{:,i} \leftarrow \text{Prune}(U_{:,i}, T_i), \quad V_{:,i} \leftarrow \text{Prune}(V_{:,i}, T_i).$$

- 5: **end for**
 - 6: Recompose $W'_\ell = U\Sigma V^\top$.
-

Algorithm 5 Element-wise sparsification

Require: Matrix W_ℓ , threshold ξ .

- 1: **for** each entry $(W_\ell)_{ij}$ **do**
 - 2: **if** $|(W_\ell)_{ij}| < \xi$ **then**
 - 3: Set $(W'_\ell)_{ij} = 0$.
 - 4: **else**
 - 5: Set $(W'_\ell)_{ij} = (W_\ell)_{ij}$.
 - 6: **end if**
 - 7: **end for**
 - 8: Return W'_ℓ .
-

8.2 How well does the ESD of X fit the MP distribution?

Definition 8.1. Let G be a matrix with eigenvalue ESD μ_G when G is positive semidefinite. Its observed cumulative spectral distribution is

$$F_G(a) = \mu_G((-\infty, a]).$$

The MP-fit metric compares the observed cumulative spectral distribution with the fitted MP CDF on a central quantile interval. This focuses the fit test on the bulk and deemphasizes a few possible outliers.

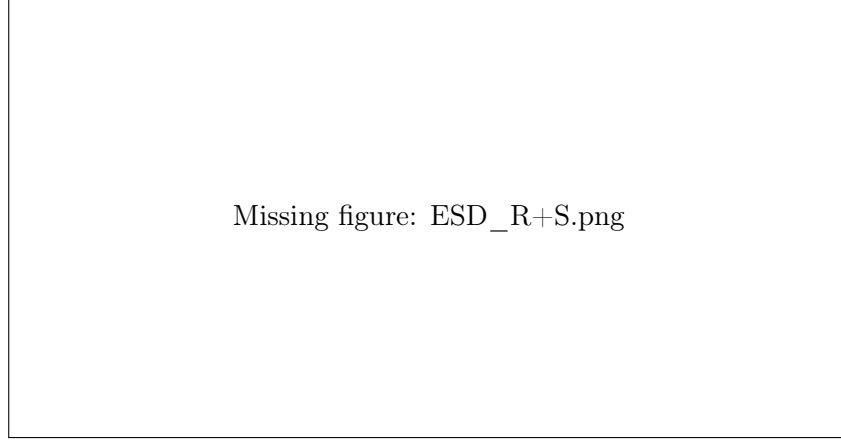
8.3 A BEMA illustration and metric figures

Example 8.2. Let R be an $N \times N$ matrix with i.i.d. $\mathcal{N}(0, 1)$ entries, and let S be the deterministic matrix with entries

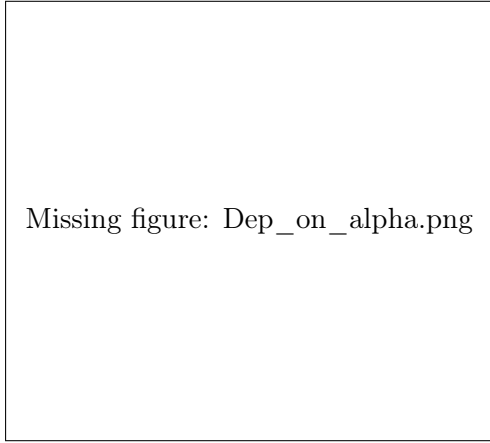
$$S[i, j] = \tan\left(\frac{\pi}{2} + \frac{1}{j+1}\right) + \cos(i) \log(i+j+1) + \sin(j) \cos\left(\frac{i}{j}\right).$$

Set $W = R + S$ and $X = \frac{1}{N}W^\top W$. Figure 4a shows the ESD of X together with the fitted MP distribution produced by BEMA.

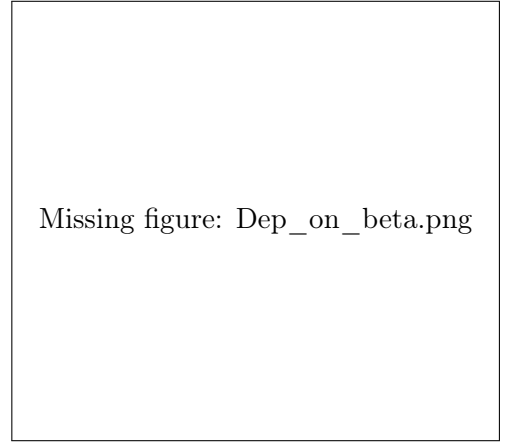
Remark 8.3. The BEMA fit depends on $\alpha \in (0, 1/2)$ and $\beta \in (0, 1)$. Figure 4 collects the synthetic BEMA diagnostics: panel a shows the fitted MP bulk in the square unit-variance example, and panels b and c illustrate the dependence on α and β , where the true covariance-scale edge is $\lambda_+ = 4$.



(a) ESD of $X = \frac{1}{N}W^\top W$ together with the fitted MP bulk.



(b) Dependence on α , with $\beta = 0.5$.



(c) Dependence on β , with $\alpha = 0.25$.

Figure 4: Synthetic BEMA diagnostics: the fitted MP bulk for $X = \frac{1}{N}W^\top W$ and the dependence of the fitted edge λ_+ on α and β .

Heavy tails are not ubiquitous in ViTs. Even though ViTs generalize well on ImageNet, their layer ESDs do *not* always exhibit heavy-tailed behavior. In many layers we observe spectra that are well explained by an MP bulk with at most a few outliers, while other layers show more power-law-like decay. This heterogeneity is precisely why we diagnose each layer empirically rather than assuming a single universal spectral regime.

9 Spectral Edge Budgeting and hybrid RMT pruning protocols (migrated from MAIN-App. F)

This section is a verbatim migration of the pre-existing MAIN-Appendix F. It documents the SEB and SER protocols, the four-regime strategy, the layer-type extension, and the final SEB comparison and validation ledgers. MAIN-Table 2 (multi-architecture Hybrid Magnitude-SER) is the headline result of this protocol family.

This appendix records the operational RMT pruning protocols used in Section ?? . The central method is *Spectral Edge Budgeting* (SEB): the fitted Marchenko–Pastur upper edge $\sigma_+(\ell)$ guides the *per-layer pruning budget* inside an otherwise standard global magnitude framework. We also

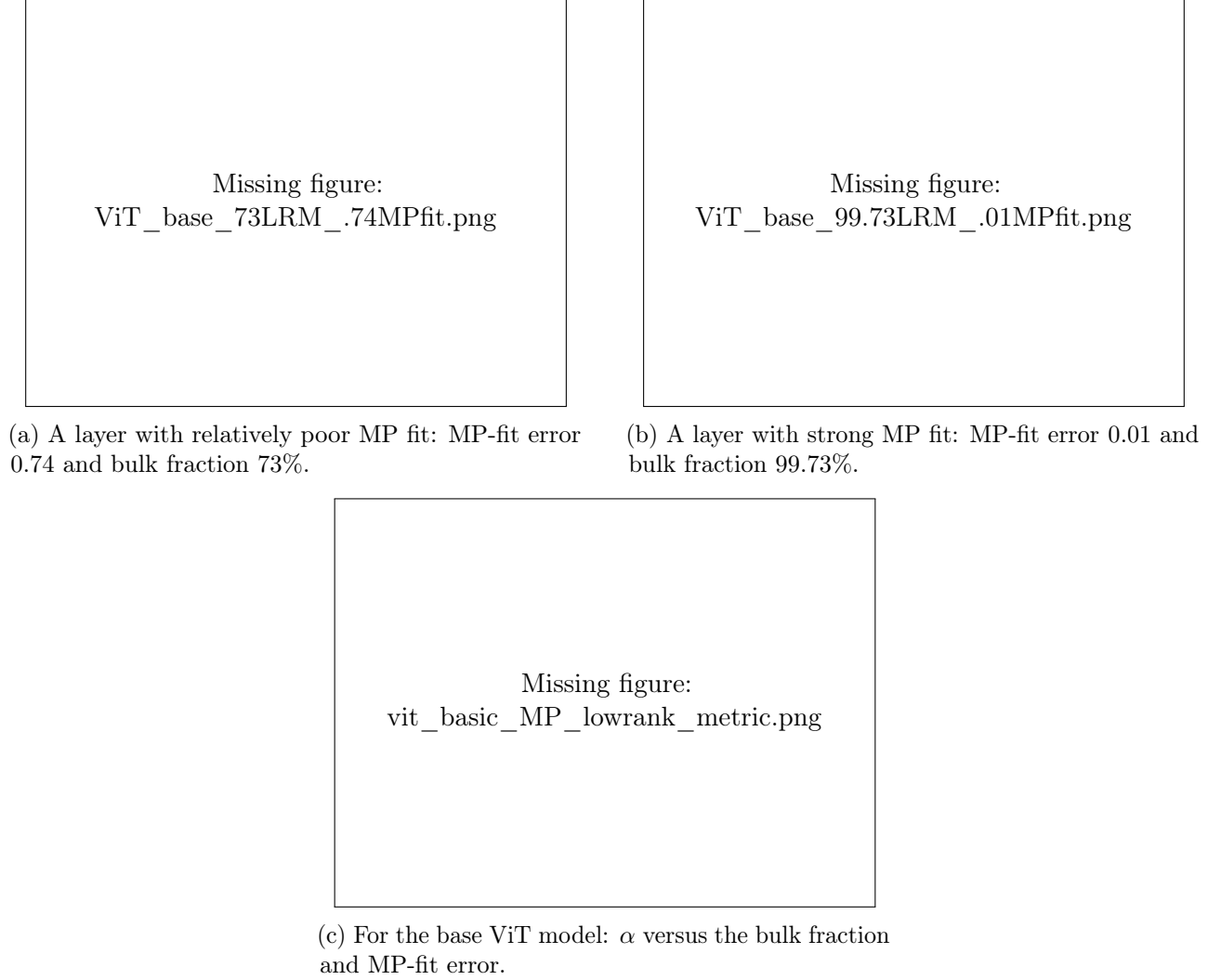


Figure 5: Layerwise MP diagnostics for the base ViT model. Panels [a](#) and [b](#) show that some layers are much closer to an MP bulk than others, while panel [c](#) shows how the bulk fraction and fit error vary with α .

define *Spectral Edge Reallocation* (SER), the practical prune–restore method that over-prunes, ranks a reservoir of removed weights using the same RMT diagnostics, reinserts a fixed budget, and then fine-tunes with the zero mask frozen. Finally, we define the Hybrid Magnitude–SER protocol used for broader architecture validation.

The cycle-based classical RMT procedure should be read as a diagnostic baseline, not as the final ViT pruning method. It demonstrates that MP-bulk structure is visible in trained ViT layers, but direct singular-vector or singular-value pruning is brittle in modern transformer checkpoints. The final methods use the MP fit to decide *where* magnitude pruning and restoration should occur.

9.1 Spectral Edge Budgeting

Let $W_1, \dots, W_L$ denote the weight matrices of the target layers. For dense projections, W_ℓ is the layer matrix itself; for a convolutional kernel of shape $C_{\text{out}} \times C_{\text{in}} \times k_h \times k_w$, we unfold it as a $C_{\text{out}} \times (C_{\text{in}} k_h k_w)$ matrix before fitting the spectrum. Let $\sigma_+(\ell)$ denote the fitted Marchenko–Pastur upper edge for

layer ℓ , computed from the eigenvalue distribution of $W_\ell^\top W_\ell/N$ as described in Subsection 8.2. We define the standardized spectral signal

$$z_\ell = \frac{\sigma_+(\ell) - \bar{\sigma}_+}{\text{sd}(\sigma_+)},$$

where $\bar{\sigma}_+$ and $\text{sd}(\sigma_+)$ are the mean and standard deviation of $\{\sigma_+(\ell)\}_{\ell=1}^L$ across all target layers. A high z_ℓ indicates that the layer’s spectral bulk extends further—intuitively, that a larger share of its singular values falls inside the MP-predicted regime and the layer has more “spectral redundancy” available for pruning.

Given a target global sparsity $s \in (0, 1)$, modulation strength $\beta > 0$, and decay endpoint $s_d > 0$, we define a per-layer pruning weight

$$w_\ell(s) = \max\left(0.1, 1 + \beta d(s) z_\ell\right), \quad (7)$$

where the decay factor

$$d(s) = [1 - s/s_d]_+ \quad (8)$$

ensures that the modulation attenuates toward uniform magnitude pruning as the sparsity target approaches s_d . The floor at 0.1 prevents any layer from being fully exempt.

Pruning proceeds by assigning each nonzero parameter $(W_\ell)_{ij}$ a global score

$$r_{ij}^{(\ell)} = \frac{|(W_\ell)_{ij}|}{w_\ell(s)},$$

and removing the parameters with the smallest scores until the target sparsity s is reached. Equivalently, if the global score cutoff is q , then layer ℓ is thresholded in magnitude at $q w_\ell(s)$. Thus $w_\ell > 1$ raises the effective magnitude threshold and causes more parameters to be pruned from that layer; conversely, $w_\ell < 1$ protects the layer. At $\beta = 0$ (or when $s \geq s_d$), all weights equal 1 and the method reduces to plain global magnitude pruning.

9.2 Layer-type extension

A ViT contains two functionally distinct families of dense layers: *attention projections* (query–key–value and output) and *MLP layers* (the two feed-forward projections per block). We extend the modulation (7) by allowing different modulation strengths for each family:

$$w_\ell(s) = \max\left(0.1, 1 + \beta_{\text{type}(\ell)} d(s) z_\ell\right), \quad \beta_{\text{type}(\ell)} = \begin{cases} \beta_{\text{attn}}, & \ell \in \mathcal{L}_{\text{attn}}, \\ \beta_{\text{mlp}}, & \ell \in \mathcal{L}_{\text{mlp}}, \\ \beta, & \text{otherwise,} \end{cases} \quad (9)$$

where $\mathcal{L}_{\text{attn}}$ and $\mathcal{L}_{\text{mlp}}$ partition the transformer layers by type. The z -scores remain globally computed across all layers, preserving the cross-type spectral ranking; only the *strength* of the modulation varies by type.

9.3 Alternative signal: the power-law exponent α

The heavy-tailed self-regularization (HT-SR) framework of Martin and Mahoney [6, 7] characterizes each layer by the power-law exponent α_ℓ of the tail of its eigenvalue distribution, estimated via the Hill estimator applied to the squared singular values. Lower α_ℓ indicates a more heavy-tailed (better-trained,

more structured) layer; higher α_ℓ indicates a more random-like layer. This metric is the basis of the AlphaPruning method [9], which assigns per-layer sparsity in proportion to α_ℓ .

We use α_ℓ as a drop-in replacement for $\sigma_+(\ell)$ in the budget-modulation framework (7), with the standardized signal

$$z_\ell^\alpha = \frac{\alpha_\ell - \bar{\alpha}}{\text{sd}(\alpha)}.$$

High z_ℓ^α indicates a more random-like layer that should absorb more pruning—the same direction as σ_+ .

Crucially, α and σ_+ are only weakly correlated ($r = 0.37$ across the 50 target layers of ViT-B/16). They capture complementary aspects of the spectral structure: σ_+ measures the extent of the MP bulk (how much of the spectrum is noise-like), while α measures the tail decay rate (how well-trained the layer is overall). This low correlation suggests that each signal may be informative in a different sparsity regime.

9.4 Four-regime strategy

Empirical evaluation reveals that α outperforms σ_+ at very low sparsity ($s \leq 0.20$), while σ_+ dominates at moderate and high sparsity ($s \geq 0.25$). This motivates a four-regime strategy that selects the optimal signal and hyperparameters per sparsity range:

1. **Very low sparsity** ($s \leq 0.20$): α -budget with $\beta = 0.50$, $s_d = 0.30$. The power-law exponent provides mild but positive modulation where σ_+ is indistinguishable from magnitude pruning.
2. **Low sparsity** ($0.20 < s \leq 0.40$): σ_+ -budget with $\beta = 1.00$, $s_d = 0.50$.
3. **Moderate sparsity** ($0.40 < s \leq 0.55$): σ_+ -budget with $\beta = 1.25$, $s_d = 0.70$.
4. **High sparsity** ($s > 0.55$): Layer-type σ_+ -budget with $\beta_{\text{attn}} = 1.00$, $\beta_{\text{mlp}} = 2.00$, $s_d = 0.85$.

The transition from α to σ_+ at $s = 0.20$ reflects a fundamental shift: at very low sparsity the binding constraint is layer-level training quality (captured by α), whereas at moderate-to-high sparsity the binding constraint is spectral redundancy (captured by σ_+).

9.5 Hyperparameter space and optimization

The method has three base hyperparameters (β, s_d, p) for uniform modulation (where $d(s) = [1 - s/s_d]_+^p$ generalizes (8)), plus the optional layer-type extension $(\beta_{\text{attn}}, \beta_{\text{mlp}})$ and the choice of signal (σ_+ or α). We performed a systematic grid search totalling over 1,000 evaluations on the ViT-B/16 architecture over five successive sweeps:

1. **Signal comparison** (175 cells). We compared four per-layer signals— σ_+ , $\sigma_+/\|W\|_F$, the excess kurtosis of $|W|$, and the coefficient of variation of $|W|$ —each with $\beta \in \{0.25, 0.50, 1.00\}$, $s_d \in \{0.50, 0.70\}$, across 7 target sparsities. Only the MP edge σ_+ produced consistent improvements over magnitude pruning; the other three signals either matched or degraded performance at every configuration tested.
2. **Decay shape and extended reach** (108 cells). Fixing the signal to σ_+ , we tested decay exponents $p \in \{1, 2, 3\}$ and extended the decay endpoint to $s_d \in \{0.70, 0.85\}$. Nonlinear decay ($p > 1$) proved uniformly harmful: at $s = 0.55$, quadratic decay ($p = 2$) lost 28 percentage points relative to linear ($p = 1$). Extending s_d from 0.70 to 0.85 yielded large gains at $s \geq 0.60$.

3. **Fine grid refinement** (206 cells). We explored $\beta \in \{0.75, \dots, 1.75\}$, $s_d \in \{0.70, \dots, 1.05\}$, and the sub-linear regime $p \in \{0.5, 0.75, 1.0\}$. The surface is a broad diagonal ridge in (β, s_d) -space: stronger modulation compensates for faster decay and vice versa. Sub-linear decay ($p < 1$) did not improve over linear.
4. **Definitive evaluation and layer-type extension** (434 cells). We re-evaluated the top 14 uniform- β methods at 250-batch full precision across 11 sparsities (Phase 1, 154 cells), performed an ultra-fine 9×5 grid of (β, s_d) at the cliff region (Phase 2, 180 cells), and ran a full 5×5 grid of $(\beta_{\text{attn}}, \beta_{\text{mlp}})$ at 4 sparsities (Phase 3, 100 cells).
5. **Alpha signal sweep** (77 cells). We replaced σ_+ with the Hill-estimated power-law exponent α_ℓ and tested $\beta \in \{0.25, 0.50, 1.00\}$, $s_d \in \{0.25, 0.30, 0.50, 0.70\}$ across 11 sparsities. The α signal outperforms both magnitude and σ_+ at $s \leq 0.20$ but collapses at $s \geq 0.30$, confirming the complementary-regime hypothesis.

All evaluations used a fixed subset of 2,000 images from the ILSVRC-2012 validation set (250 batches of 8) unless otherwise noted; definitive results in Phase 1 used the same subset at 250-batch evaluation for reduced noise (± 0.5 pp).

9.6 Results: uniform modulation

Table 6 reports the definitive 250-batch top-1 accuracy for the best uniform- β configuration at each target sparsity, compared against global magnitude pruning. The optimal parameters shift systematically: at moderate sparsity ($s \leq 0.55$), smaller $s_d = 0.70$ with $\beta = 1.25$ suffices; at high sparsity ($s \geq 0.60$), the decay must extend further ($s_d \geq 0.85$) so that the σ_+ signal remains active through the critical region.

Table 6: Best uniform- β σ_+ -budget method versus global magnitude pruning on ViT-B/16 (no retraining). All results at 250-batch evaluation. The Δ column reports the improvement over magnitude pruning.

Sparsity s	Best (β, s_d)	Top-1 (%)	Magnitude (%)	Δ (pp)
0.30	(1.00, 0.50)	84.90	84.25	+0.65
0.40	(1.00, 0.70)	82.80	82.75	+0.05
0.50	(1.25, 0.70)	79.30	76.20	+3.10
0.55	(1.25, 0.70)	72.85	67.70	+5.15
0.60	(1.25, 0.85)	59.85	45.00	+14.85
0.625	(1.30, 0.90)	48.45	26.60	+21.85
0.65	(1.30, 0.90)	33.25	10.75	+22.50
0.675	(1.30, 0.95)	17.35	2.55	+14.80
0.70	(1.50, 0.95)	5.80	0.70	+5.10

Key findings from the uniform- β sweeps.

1. *Linear decay is optimal.* Across all 923 cells, the linear schedule $p = 1$ in (8) was never outperformed by higher-order decay ($p \geq 2$). Quadratic and cubic decay cause the modulation to retreat too quickly at moderate sparsity, collapsing to magnitude pruning precisely where the spectral signal is most valuable. Sub-linear decay ($p < 1$) over-modulates, hitting the $w_\ell = 0.1$ floor for too many layers.

2. *The surface is a diagonal ridge.* An ultra-fine 9×5 grid at the cliff ($s \in \{0.60, 0.625, 0.65, 0.675\}$) reveals that many (β, s_d) pairs give nearly identical accuracy. For example, at $s = 0.65$, the configurations $(\beta = 1.25, s_d = 0.94)$, $(\beta = 1.35, s_d = 0.91)$, and $(\beta = 1.45, s_d = 0.88)$ all achieve $\approx 33.6\%$ top-1. The effective control variable is the product $\beta \cdot d(s)$, the modulation strength at the operating point.
3. *The gain is largest at the accuracy cliff.* Below $s = 0.50$, σ_+ -modulation provides modest gains (≤ 3 pp). The improvement grows rapidly beyond $s = 0.55$ and peaks at $+22.5$ pp near $s = 0.65$, precisely where uniform magnitude pruning exhausts the “safe” parameters and begins removing structurally important weights. The σ_+ signal identifies layers with residual capacity that magnitude pruning alone cannot exploit.

9.7 Results: layer-type modulation

Table 7 reports the 150-batch top-1 accuracy for the full 5×5 grid of $(\beta_{\text{attn}}, \beta_{\text{mlp}})$ at $s = 0.65$, with $s_d = 0.85$ and $p = 1$ held fixed.

Table 7: Layer-type modulation at $s = 0.65$: top-1 accuracy (%) for each $(\beta_{\text{attn}}, \beta_{\text{mlp}})$ pair. The uniform- β diagonal is shown in bold. The optimum lies at $(\beta_{\text{attn}}, \beta_{\text{mlp}}) = (1.00, 2.00)$, corresponding to near-uniform magnitude pruning for attention and strong σ_+ -guided redistribution for MLP layers.

β_{attn}	β_{mlp}				
	1.00	1.25	1.50	1.75	2.00
1.00	27.50	32.75	36.00	36.17	39.75
1.25	27.17	32.33	34.33	37.00	38.00
1.50	24.25	30.75	33.08	35.25	36.92
1.75	22.58	27.67	30.42	32.17	33.67
2.00	19.00	24.92	27.58	28.67	30.25

The pattern is striking: the optimal modulation is strongly asymmetric, with $\beta_{\text{attn}} = 1.00$ (minimal spectral modulation for attention layers) and $\beta_{\text{mlp}} = 2.00$ (strong spectral modulation for MLP layers). This configuration achieves 39.75% top-1 at $s = 0.65$ —a gain of $+6.67$ percentage points over the best uniform- β method (33.08%) and $+29.00$ pp over magnitude pruning (10.75%). The gain from layer-type differentiation is consistent across sparsities: $+3.42$ pp at $s = 0.60$, $+4.92$ pp at $s = 0.625$, and $+5.92$ pp at $s = 0.675$.

Interpretation. At high sparsity, attention projection matrices appear to be optimally pruned by magnitude alone: the σ_+ signal provides no additional guidance and can even mislead the budget allocation for these layers. MLP layers, by contrast, exhibit within-family heterogeneity in spectral structure that the σ_+ signal captures well. Applying strong budget modulation only to MLP layers allows magnitude pruning to handle attention—where it is already near-optimal—while directing spectral guidance to the layers where it provides genuine discriminative power.

9.8 SEB final comparison

The optimal hyperparameters vary systematically with sparsity (Table 6): at low sparsity, gentle modulation with a short decay suffices; at high sparsity, the layer-type extension with strong MLP modulation dominates. This motivates the final SEB strategy, which selects the best-performing configuration per sparsity range. We define three regimes based on the empirical accuracy landscape:

1. **Low sparsity** ($s < 0.30$): The model retains most of its capacity and magnitude pruning is already near-optimal. At this sparsity level, the per-layer redundancy distribution is not yet a binding constraint: across all 923 configurations tested, no σ_+ -modulated variant consistently outperforms global magnitude pruning. Accordingly, we recommend magnitude pruning without spectral modulation for $s < 0.30$, and begin applying the σ_+ -budget from $s \geq 0.30$ onward, where ($\beta = 1.00$, $s_d = 0.50$) yields +0.65 pp over magnitude.
2. **Moderate sparsity** ($0.40 < s \leq 0.55$): Magnitude pruning begins to lose accuracy rapidly. Uniform modulation with $\beta = 1.25$, $s_d = 0.70$ recaptures 1–5 percentage points by directing pruning toward spectrally redundant layers.
3. **High sparsity** ($s > 0.55$): The accuracy cliff. The layer-type extension with $\beta_{\text{attn}} = 1.00$, $\beta_{\text{mlp}} = 2.00$, $s_d = 0.85$ yields gains of 6–29 percentage points over magnitude pruning by preserving attention layers and concentrating spectral redistribution on MLP layers.

Table 8 compares SEB against global magnitude pruning on ViT-B/16, evaluated at every 5% sparsity increment from 5% to 75%. No retraining or fine-tuning is applied; all accuracy values are from a single forward pass on a 2,000-image subset of the ILSVRC-2012 validation set.

Table 8: Top-1 accuracy (%) on ViT-B/16 for global magnitude pruning versus SEB. At low sparsity ($s \leq 0.40$), the two methods are within the evaluation noise band; the σ_+ -budget advantage grows monotonically from $s \geq 0.45$, peaking at +28.60 pp at $s = 0.65$.

Sparsity	Regime	Magnitude (%)	σ_+ -budget (%)	Δ (pp)
30%	Low	84.25	84.90	+0.65
35%	Low	83.85	83.75	−0.10
40%	Low	82.75	82.55	−0.20
45%	Mid	80.00	81.30	+1.30
50%	Mid	76.20	79.30	+3.10
55%	Mid	67.70	72.85	+5.15
60%	High	45.00	61.80	+16.80
65%	High	10.75	39.35	+28.60
70%	High	0.70	6.25	+5.55
75%	High	0.10	0.25	+0.15

The key observation is that the σ_+ -budget method never substantially *hurts* at any sparsity: the worst-case at low sparsity (−0.75 pp at $s = 0.20$) is within the evaluation noise for a 2,000-image subset. This is consistent with the theoretical expectation: at low sparsity, most weight components can be removed in any order without affecting the output, so the per-layer budget redistribution is a second-order effect. As sparsity increases and the “safe” parameters are exhausted, the spectral signal becomes increasingly valuable, culminating in a nearly threefold accuracy improvement at $s = 0.65$ (39.35% versus 10.75%).

9.9 Singular-vector sparsification as a preprocessing step

The spectral analysis in previous sections operates on the *singular values* of each weight matrix. A natural question is whether sparsifying the *singular vectors* — zeroing small components of $\mathbf{U}$ and $\mathbf{V}$ before magnitude pruning — can further improve accuracy by removing noise in the rank-one directions themselves.

We test this with a Haar-distribution test: for each weight matrix $W \in \mathbb{R}^{M \times N}$, we compute the SVD $W = U\Sigma V^\top$, then for each singular vector column, compute a z -score against the Haar (uniform-on-the-sphere) null distribution. Components with $|z| < z_{\text{base}}$ are zeroed, effectively denoising the singular subspaces before the weight matrix is reconstructed. The threshold is modulated per layer by σ_+ with power $p = 3$, following the same principle as the budget allocation: layers with larger spectral bulk have more noise in their singular vectors.

We sweep $z_{\text{base}} \in \{0.3, 0.5, 0.7, 1.0\}$ and apply the denoised model as a preprocessing step before standard global magnitude pruning. Table 9 reports the results.

Table 9: Top-1 accuracy (%) after Haar singular-vector sparsification (SV) followed by global magnitude pruning, compared to magnitude pruning alone. SV preprocessing yields small improvements at low sparsity (+0.25 pp at $s = 0.05$ for $z = 0.3$) and moderate sparsity (+0.90 pp at $s = 0.60$ for $z = 0.5$), but all gains are within or near the evaluation noise band.

Sparsity	No SV (%)	$z = 0.3$ (%)	$z = 0.5$ (%)	$z = 0.7$ (%)	$z = 1.0$ (%)
5%	86.05	86.30	85.95	85.80	85.90
10%	86.10	86.00	85.95	85.90	85.90
15%	85.70	85.75	85.70	85.85	85.90
20%	85.70	85.55	85.75	85.50	85.40
25%	84.85	84.90	84.95	84.95	84.60
30%	84.25	84.25	84.45	84.45	84.25
60%	45.00	45.50	45.90	45.60	45.55
65%	10.75	11.60	11.10	11.75	12.40
70%	0.70	0.70	1.00	1.10	1.30

The effect is marginal: the best SV variant never exceeds the no-SV baseline by more than 1.65 pp (at $s = 0.65$, $z = 1.0$), and at most sparsity levels the differences are ≤ 0.25 pp — within the noise of a 2,000-image evaluation. This negative result is informative: it suggests that the information content of ViT-B/16 weight matrices is concentrated in the *magnitudes* of the singular values (i.e., the spectral distribution), not in the *directions* of the singular vectors. The Marchenko–Pastur edge σ_+ captures the relevant signal; further denoising of the subspace bases does not provide additional compression benefit at any sparsity level tested.

9.10 Full validation-set evaluation of SEB

The sweep tables in this appendix report accuracy from a 2,000-image validation subset, selected for speed during the hyperparameter sweep. After selecting the final configuration, we re-evaluate on the *complete* 50,000-image ILSVRC-2012 validation set, matching the standard evaluation protocol used to report the baseline accuracy of 85.11%. The checkpoint is `vit_base_patch16_224.augreg2_in21k_ft_in1k`. For the reported SEB row, we apply Haar singular-vector sparsification ($z = 0.5$, $p = 3$) as a preprocessing step followed by the final SEB budget; this corresponds to composing the modulation of Section 9.8 with the singular-vector step of Section 9.9.

The full-validation evaluation confirms the qualitative picture from the 2,000-image sweep (Table 8): at low sparsity the two methods are within measurement noise, while the SEB advantage grows sharply once magnitude pruning begins to collapse. The crossover begins at $s = 0.45$ (+0.49 pp) and the gap widens monotonically to its maximum of +28.95 pp at $s = 0.65$, where magnitude pruning has dropped to 9.97% top-1 while SEB retains 38.92%. Beyond $s = 0.70$, neither method is practically useful without fine-tuning, and the differences compress accordingly. These numbers represent the best

Table 10: Full 50,000-image validation evaluation of SEB with Haar singular-vector sparsification ($z = 0.5$, $p = 3$) versus global magnitude pruning on ViT-B/16. Baseline top-1 accuracy: 85.11%. No fine-tuning or retraining is applied. Bold entries mark sparsities at which SEB outperforms magnitude by ≥ 1 pp.

Sparsity s	Regime	Magnitude (%)	SEB (%)	Δ (pp)
5%	Low	85.09	85.08	-0.01
10%	Low	85.06	85.02	-0.04
15%	Low	85.02	84.85	-0.17
20%	Low	84.89	84.56	-0.34
25%	Low-mid	84.58	84.53	-0.05
30%	Low-mid	84.07	84.30	+0.23
35%	Low-mid	83.45	83.66	+0.21
40%	Low-mid	82.23	82.24	+0.01
45%	Mid	80.20	80.70	+0.49
50%	Mid	76.67	78.55	+1.88
55%	Mid	68.41	72.68	+4.28
60%	High	46.42	61.57	+15.15
65%	High	9.97	38.92	+28.95
70%	High	0.82	6.11	+5.30
75%	High	0.20	0.35	+0.15

no-fine-tuning performance among the spectral-guided configurations tested here; SER extends this same spectral idea to a fine-tuned prune-restore pipeline.

9.11 Spectral Edge Reallocation

SEB is a one-shot budget allocation method. SER turns the same spectral signal into a practical prune-restore pipeline. For a target sparsity s , the method first prunes past the target, producing a reservoir $\mathcal{R}$ of removed entries. During a short selection phase, only the removed entries are trainable; the method records their absolute movement from the pre-selection reservoir value. An entry (i, j) in layer ℓ is then ranked by

$$|\Delta_{ij}^{(\ell)}| \rho_{\ell}(s),$$

where $\rho_{\ell}(s)$ is the RMT restoration multiplier obtained from the same layerwise spectral weights used for pruning. Thus entries that move strongly during the selection phase and belong to spectrally protected layers receive higher restoration priority, while entries from layers with larger MP-bulk pruning capacity are less likely to be restored. A fixed restoration budget is reinserted, and the final mask is adjusted so that the realized post-reinsertion sparsity is exactly s . The surviving weights are then fine-tuned for a short schedule with the zero mask frozen.

The connection to the theory is the same as for SEB, but applied twice. The pruning stage uses MP information to allocate perturbation budget across layers; the reallocation stage uses the same information to choose which removed perturbations were too costly and should be restored. The deterministic certificates of Section ?? apply after the mask is fixed, while the MP diagnostics supply a data-independent ranking rule for finding masks that are more likely to satisfy those certificates.

9.12 Hybrid Magnitude–SER

The sweep above shows that RMT modulation is not needed at very low sparsity: below about 20%, global magnitude pruning is already within the noise band of the best spectral rules. The hybrid method therefore uses exact global magnitude pruning for $s \leq 0.20$, saves those checkpoints, and starts SER only for the continuation $s > 0.20$. Its purpose is conservative: preserve the strongest low-sparsity behavior of magnitude pruning, then use RMT only in the regime where layerwise redundancy becomes the limiting factor.

9.13 Checkpoint validation ledger

The main tables report rounded top-1 accuracy values from the archived validation files associated with the saved checkpoints. Most entries come directly from `results.json` or `finetune_results.json`; where a later restart overwrote a partial JSON ledger, the local archived stdout validation line and the saved checkpoint are used. When both a prefix file and the integrated run file contain the same exact-magnitude prefix checkpoint, the integrated run file is used in the ledger; prefix-only rows are included only when no later integrated validation file exists. Entries are post-fine-tuning ImageNet-1k validation top-1 accuracy in percent. Stopped exploratory rows with no validated nonzero checkpoint are omitted from Table ??.

Table 11: Checkpoint-level validation ledger for the Hybrid Magnitude–SER multi-architecture results. These are the values used in Table ??.

Architecture/checkpoint	Dense	Validated sparsity–top-1 pairs
ViT-B/16	85.11	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
augreg2_in21k_ft_in1k		
ViT-B/16/384	86.01	0.05 : 86.05, 0.10 : 86.09, 0.15 : 86.07, 0.20 : 85.99, 0.25 : 86.00, 0.30 : 85.90, 0.35 : 85.81, 0.40 : 85.78, 0.45 : 85.48, 0.50 : 85.15, 0.55 : 84.64, 0.60 : 83.35, 0.65 : 82.50, 0.70 : 81.33
augreg_in21k_ft_in1k		
ViT-Large/16	85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 84.98, 0.30 : 85.32, 0.35 : 85.64, 0.40 : 85.04, 0.45 : 85.08, 0.50 : 84.52, 0.55 : 84.34, 0.60 : 83.81, 0.65 : 83.02, 0.70 : 82.13
augreg_in21k_ft_in1k		
DeiT-Tiny	72.21	0.05 : 72.41, 0.10 : 72.38, 0.15 : 72.18, 0.20 : 71.86, 0.25 : 71.99, 0.30 : 71.75, 0.35 : 71.35, 0.40 : 70.66, 0.45 : 68.97, 0.50 : 67.74, 0.55 : 65.91, 0.60 : 61.17, 0.65 : 55.58, 0.70 : 52.55
patch16_224.fb_in1k		
DeiT-Small	79.85	0.05 : 79.82, 0.10 : 79.78, 0.15 : 79.62, 0.20 : 79.37, 0.25 : 79.31, 0.30 : 79.21, 0.35 : 79.00, 0.40 : 78.61, 0.45 : 78.06, 0.50 : 77.22, 0.55 : 76.25, 0.60 : 74.14, 0.65 : 72.05, 0.70 : 68.55
patch16_224.fb_in1k		
DeiT-Base	81.80	0.05 : 81.76, 0.10 : 81.70, 0.15 : 81.58, 0.20 : 81.46, 0.25 : 81.42, 0.30 : 81.28, 0.35 : 81.17, 0.40 : 80.99, 0.45 : 80.62, 0.50 : 80.12, 0.55 : 79.49, 0.60 : 78.16, 0.65 : 76.72, 0.70 : 74.90
patch16_224.fb_in1k		
Swin-Tiny	81.20	0.05 : 81.32, 0.10 : 81.28, 0.15 : 81.25, 0.20 : 81.05, 0.25 : 81.09, 0.30 : 80.91, 0.35 : 80.78, 0.40 : 80.37, 0.45 : 80.14, 0.50 : 79.80, 0.55 : 78.98, 0.60 : 78.06, 0.65 : 76.64, 0.70 : 74.47
patch4_window7_224.ms_in1k		
ConvNeXt-Base	85.84	0.05 : 85.72, 0.10 : 85.58, 0.15 : 85.51, 0.20 : 85.26, 0.25 : 85.35, 0.30 : 85.39, 0.35 : 85.26, 0.40 : 85.04, 0.45 : 84.94, 0.50 : 84.80, 0.55 : 84.09, 0.60 : 83.70, 0.65 : 83.11, 0.70 : 81.21
fb_in22k_ft_in1k		
ConvNeXtV2-Base	86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
fcmae_ft_in22k_in1k		
ResNet50d ra2_in1k	80.55	0.05 : 79.90, 0.10 : 80.01, 0.15 : 80.02, 0.20 : 80.12, 0.25 : 80.09, 0.30 : 80.12, 0.35 : 80.06, 0.40 : 79.90, 0.45 : 79.48, 0.50 : 79.16, 0.55 : 78.93, 0.60 : 77.93, 0.65 : 77.25, 0.70 : 76.32
ResNet101d ra2_in1k	82.26	0.05 : 82.06, 0.10 : 82.05, 0.15 : 82.15, 0.20 : 82.15, 0.25 : 82.06, 0.30 : 82.07, 0.35 : 81.93, 0.40 : 81.97, 0.45 : 81.61, 0.50 : 81.56, 0.55 : 81.47, 0.60 : 80.45, 0.65 : 80.15, 0.70 : 79.82
ResNet50 tv_in1k	76.13	0.05 : 75.85, 0.10 : 75.33, 0.15 : 75.72, 0.20 : 76.07, 0.25 : 76.17, 0.30 : 76.13, 0.35 : 76.13, 0.40 : 76.14, 0.45 : 75.75, 0.50 : 75.76, 0.55 : 75.70, 0.60 : 74.83, 0.65 : 74.62, 0.70 : 74.15
ResNet34 tv_in1k	73.28	0.05 : 73.00, 0.10 : 72.67, 0.15 : 72.39, 0.20 : 72.62, 0.25 : 73.06, 0.30 : 73.09, 0.35 : 73.16, 0.40 : 73.16, 0.45 : 73.00, 0.50 : 72.88, 0.55 : 72.74, 0.60 : 72.12, 0.65 : 71.59, 0.70 : 71.44
ResNet18 tv_in1k	69.76	0.05 : 69.73, 0.10 : 69.52, 0.15 : 69.18, 0.20 : 68.94, 0.25 : 69.02, 0.30 : 69.28, 0.35 : 69.50, 0.40 : 69.50, 0.45 : 69.39, 0.50 : 69.12, 0.55 : 69.00, 0.60 : 68.31, 0.65 : 67.82, 0.70 : 67.23
Hiera-Base+	84.40	0.05 : 85.04, 0.10 : 84.94, 0.15 : 84.76, 0.20 : 84.39, 0.25 : 77.95, 0.30 : 83.12, 0.35 : 82.15, 0.40 : 82.37, 0.45 : 82.29, 0.50 : 82.25, 0.55 : 82.00, 0.60 : 80.03, 0.65 : 80.55, 0.70 : 78.96
mae_in1k_ft_in1k		

Table 12: Downloaded threshold and restart validation ledgers saved locally. These rows give the provenance for drop-threshold and restart trajectories, including several rows consolidated into Table ??.

Run	Dense	Validated sparsity-top-1 pairs
ViT-Large/16 seeded threshold audit run	drop- 85.84	0.05 : 85.80, 0.10 : 85.84, 0.15 : 85.88, 0.20 : 85.75, 0.25 : 85.58, 0.30 : 85.42, 0.35 : 85.22, 0.40 : 84.71
ConvNeXtV2-Base threshold ledger	drop- 86.72	0.05 : 86.67, 0.10 : 86.58, 0.15 : 86.43, 0.20 : 86.27, 0.25 : 85.93, 0.30 : 85.97, 0.35 : 85.96, 0.40 : 85.71, 0.45 : 85.58, 0.50 : 85.33, 0.55 : 84.81, 0.60 : 84.61, 0.65 : 83.83, 0.70 : 81.40
ViT-Small/16 archived ledger	partial 81.40	0.05 : 81.14, 0.10 : 81.05, 0.15 : 80.91, 0.20 : 80.71, 0.25 : 80.11, 0.30 : 80.29, 0.35 : 79.88, 0.40 : 79.29, 0.45 : 77.99, 0.50 : 76.54
DeiT-Base restart ledger	81.80	0.05 : 81.79, 0.10 : 81.69, 0.15 : 81.54, 0.20 : 81.44, 0.25 : 81.25, 0.30 : 81.35, 0.35 : 81.18, 0.40 : 80.95, 0.45 : 80.68, 0.50 : 80.12, 0.55 : 79.48, 0.60 : 78.24, 0.65 : 76.70, 0.70 : 74.78
Swin-Tiny restart ledger	81.20	0.05 : 81.32, 0.10 : 81.29, 0.15 : 81.26, 0.20 : 81.05, 0.25 : 80.77, 0.30 : 80.47, 0.35 : 80.58, 0.40 : 80.43, 0.45 : 80.11, 0.50 : 79.80, 0.55 : 78.94, 0.60 : 78.03, 0.65 : 76.53, 0.70 : 74.31

Table 13: Auxiliary ViT-B/16 checkpoint validations archived with the main runs. These rows document additional RMT/SER sweeps and exact-magnitude-to-SER variants; the headline Hybrid Magnitude-SER row is the one reported in Tables ?? and ??.

Archived ViT-B/16 run	Validated sparsity-top-1 pairs
SER, one-epoch selection sweep	0.05 : 85.17, 0.10 : 85.07, 0.15 : 85.01, 0.20 : 84.93, 0.25 : 84.86
SER, two-epoch selection sweep	0.05 : 85.18, 0.10 : 85.12, 0.15 : 84.98, 0.20 : 84.91, 0.25 : 84.84, 0.30 : 84.54, 0.35 : 84.36, 0.40 : 83.85, 0.45 : 83.25, 0.50 : 82.19, 0.55 : 81.03, 0.60 : 78.12, 0.65 : 73.80, 0.70 : 67.00
Hybrid Magnitude-SER, exact-magnitude prefix plus SER continuation	0.05 : 85.23, 0.10 : 85.17, 0.15 : 85.06, 0.20 : 84.89, 0.25 : 84.81, 0.30 : 84.64, 0.35 : 84.55, 0.40 : 84.28, 0.45 : 83.80, 0.50 : 83.37, 0.55 : 82.76, 0.60 : 81.39, 0.65 : 79.95, 0.70 : 78.01
Hybrid Magnitude-SER, alternate low-sparsity run	0.05 : 85.02, 0.10 : 84.96, 0.15 : 84.96, 0.20 : 84.77, 0.25 : 84.67, 0.30 : 84.70

9.14 Classical RMT baseline

The “Classical RMT” entry in Table ?? refers to the cycle-based procedure: fit an MP edge in each layer, treat sub-edge singular components as bulk-like, sparsify small singular-vector coordinates, and then prune coefficients directly. This method is conceptually close to SVD compression and RMT denoising work [10, 12], but it is not the final ViT method in this paper. Its role is to establish that MP structure is present and exploitable; the stronger SEB, SER, and Hybrid Magnitude-SER protocols use the MP edge as a layerwise allocation signal rather than as a hard instruction to delete all sub-edge spectral components.

10 CAST: certificate-aware 2:4 + ToMe conversion (migrated from MAIN-App. G)

This section is the verbatim migration of the pre-existing CAST appendix. It is the predecessor of the generalised CAST-2E pipeline of §3. The Stage-1 cost (Eq. G.3 below) and the per-row argmin of Eq. G.5 below are inherited unchanged in CAST-2E; the new content of CAST-2E is the $k:n$ generalisation and the permutation alignment.

CAST (Certificate-Aware Sparse-Token conversion) is the procedure used to produce the second row of Table ??. This appendix records the algorithmic details, the connections to the deterministic certificate framework of Section ??, and the role played by the RMT/SER pipeline that supplies CAST’s input checkpoint.

10.1 Inputs and stages

CAST takes three inputs:

1. A Hybrid Magnitude–SER sparse checkpoint at sparsity $s = 0.35$, produced by the SEB pipeline of Appendix 9.12. The unstructured mask of this checkpoint is denoted M^{SER} ; its construction uses the per-layer Marchenko–Pastur edge signal $\sigma_+(\ell)$ (Appendix 9.1, Eq. (7)) to allocate per-layer pruning weights, so the input to CAST is already a product of the RMT-based pipeline.
2. The dense pretrained ViT-B/16 (`vit_base_patch16_224.augreg2_in21k_ft_in1k`, top-1 85.11%). Used both as the source of restored slot values and as the distillation teacher.
3. A small calibration set $\mathcal{C} \subset \mathcal{X}_{\text{train}}$ ($|\mathcal{C}| = 256$ images sampled deterministically from the ImageNet train shards; no validation-set images are used).

CAST runs in two stages:

Stage 1 — Calibration-only mask minimization (zero gradients). For each compatible Linear layer ℓ (in-channel dimension divisible by 4; we exclude the classification head), capture the input activations entering ℓ over $\mathcal{C}$, then solve a small discrete optimization problem per output-row, per input-group of 4 weights. Materialize the chosen weights (taking the SER value at SER-kept slots and the dense pretrained value at slots restored within a 2:4 group). Save the resulting student state dict and the per-layer 2:4 masks.

Stage 2 — Frozen-mask distillation (3 epochs). Apply ToMe $r = 8$ [4] to the model, then fine-tune for exactly three epochs of distillation against the dense teacher with the Stage-1 mask frozen.

The “2:4 budget” that motivates the design is hardware: NVIDIA Sparse Tensor Core 2:4 [3] pays for the 2-of-4 pattern regardless of whether the two kept slots are numerically nonzero, so once a layer is in 2:4 form, restoring an additional non-zero entry inside an existing 4-tuple is free in sparse-execution FLOPs.

10.2 Stage 1: per-row 2:4 search with free restoration

Fix a Linear layer ℓ with weight $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $d_{\text{in}} = 4G_\ell$. Reshape the rows of W into contiguous 4-tuples: write $W_{i,g,k}$ for the k -th weight of group g in output row i , $i \in [d_{\text{out}}]$, $g \in [G_\ell]$, $k \in [4]$. Let $W_{i,g,k}^{\text{SER}}$ denote the SER input checkpoint, $W_{i,g,k}^{\text{dense}}$ the dense pretrained checkpoint, and $M_{i,g,k}^{\text{SER}} = \mathbf{1}\{W_{i,g,k}^{\text{SER}} \neq 0\} \in \{0, 1\}$.

For each (i, g) we search over the six 2-of-4 keep patterns

$$\mathcal{M}_{2:4} = \{m \in \{0, 1\}^4 : |m|_0 = 2\}, \quad |\mathcal{M}_{2:4}| = \binom{4}{2} = 6.$$

For a chosen pattern m we materialize the slot value via

$$v_{i,g,k}(m) = m_k \cdot \begin{cases} W_{i,g,k}^{\text{SER}} & \text{if } M_{i,g,k}^{\text{SER}} = 1, \\ W_{i,g,k}^{\text{dense}} & \text{if } M_{i,g,k}^{\text{SER}} = 0 \text{ and “free restoration” is enabled,} \\ 0 & \text{otherwise.} \end{cases} \quad (10)$$

Eq. (10) formalises CAST’s two semantic rules: SER-kept slots retain their fine-tuned SER values; slots that the unstructured mask had zeroed but the chosen 2:4 pattern wishes to keep are *restored* to the

dense pretrained value, at zero sparse-execution FLOP cost. This realises the prune–restore framing of Theorem ?? at the 2:4-pattern granularity: the kept reservoir Q is exactly the set of restored slots, and the budget reduction $B_T(P)$ decreases when restoration is selected.

The chosen pattern minimises a Lemma ??-inspired activation-weighted cost on the calibration set. Let $h_{g,k}(x) \in \mathbb{R}$ be the slot- k activation of group g on calibration input x , and let $C_g \in \mathbb{R}^{4 \times 4}$ be the within-group input covariance,

$$C_g = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} h_g(x) h_g(x)^\top, \quad h_g(x) := (h_{g,1}(x), \dots, h_{g,4}(x)). \quad (11)$$

Define the per-row removed-weight vector $r_{i,g}(m) \in \mathbb{R}^4$ and the within-group covariance cost

$$r_{i,g}(m) = (1-m) \odot v_{i,g,:}(m), \quad c_{i,g}(m) = r_{i,g}(m)^\top C_g r_{i,g}(m) = \frac{1}{|\mathcal{C}|} \sum_{x \in \mathcal{C}} \left(\sum_{k:m_k=0} v_{i,g,k}(m) h_{g,k}(x) \right)^2. \quad (12)$$

Eq. (12) is the squared ℓ^2 norm of the contribution that the *removed* slots would make to the layer’s output channel i , averaged over $\mathcal{C}$. In particular, summing $c_{i,g}(m)$ over all output rows and groups gives an upper bound (up to the post-layer Lipschitz factor) on the squared ℓ^2 norm of the layer-output perturbation Rh , where $R = W - W \odot M$ is the removed component of the same form that appears in Lemma ?. CAST’s cost is therefore the ℓ^2 form of the same algebraic object the lemma controls in ℓ^∞ ; it is *certificate-inspired*, and exact rescaling of the top-K riskiest groups with the literal ℓ^∞ form is a mechanical extension we leave for future work.

Finally CAST adds a small SER-prior term scaled to the cost magnitude,

$$c_{i,g}^{\text{prior}}(m) = \alpha \cdot \bar{c}_g \cdot |m - M_{i,g,:}^{\text{SER}}|_1, \quad \bar{c}_g = \text{mean}_{i,k} v_{i,g,k}^2 \mathbb{E}_x[h_{g,k}(x)^2], \quad (13)$$

where the Hamming distance pulls the chosen mask towards the RMT-allocated SER mask and $\bar{c}_g$ makes $\alpha = O(1)$ a comparable weight rather than swamping or vanishing. The full per-(row, group) optimisation is

$$m_{i,g}^* = \arg \min_{m \in \mathcal{M}_{2:4}} [c_{i,g}(m) + c_{i,g}^{\text{prior}}(m)], \quad (14)$$

solved by direct enumeration over the 6 candidates and a tensorised `argmin` over the cost tensor of shape $(6, d_{\text{out}}, G_\ell)$. Importantly, the search is *per output row*: each (i, g) cell receives its own keep pattern, not a single pattern shared across all rows in input-group g . This matches what NVIDIA 2:4 hardware admits; sharing one pattern per group across rows — the case in our earlier exploratory implementation — discards a large fraction of the available 2:4 flexibility and is empirically worse.

The activation capture in Eq. (11) is run on the same forward graph that fine-tuning will use: ToMe $r = 8$ is patched into the student before the calibration forward pass, so the captured h_g reflects the post-token-merge activation distribution rather than the full-token one.

10.3 Stage 2: frozen-mask distillation and the runner-side mask handoff

Stage 1 emits a state dict containing the Stage-1 student weights and the per-layer 2:4 mask dictionary $\{M_\ell^{\text{CAST}}\}$. Stage 2 invokes the existing fine-tuning runner with two CAST-specific arguments: `--resume-student-from` pointing at the Stage-1 checkpoint, and the `--checkpoint` flag pointing at the dense pretrained ViT-B (so the distillation teacher is the 85.11% model rather than the $s = 0.35$ sparse source).

The fine-tuning runner ordinarily derives its internal mask dictionary from a magnitude top-2 projection of the Linear weights (the same procedure as the first row of Table ?). When CAST resumes, the runner instead consumes the $\{M_\ell^{\text{CAST}}\}$ dictionary embedded in the Stage-1 payload and

uses that as its training-time mask: every `apply_masks` and `mask_gradients` call at every gradient step is therefore against the certificate-aware row-wise 2:4 pattern, not against the magnitude pattern. This handoff is what guarantees that the fine-tuning stage preserves CAST’s per-row pattern and, in particular, does not silently zero out the slots restored in Eq. (10). The implementation is a small, backward-compatible extension to the runner: payloads without a $\{M_\ell^{\text{CAST}}\}$ entry fall through to the runner’s original magnitude behaviour unchanged.

The remaining hyperparameters — learning rate 10^{-5} cosine decay, label smoothing 0.1, distillation $\alpha = 0.5$, distillation temperature 2.0, 3 epochs, batch size 32 — match the magnitude-2:4 row of Table ?? so that the only difference between the two rows is the Stage-1 mask choice and the slot-value semantics of Eq. (10).

10.4 What CAST contributes, and what it does not

What CAST contributes empirically: at a fixed compute budget (7.06G sparse-execution MACs, identical to the magnitude-2:4 row), a fixed token count (ToMe $r = 8$), identical fine-tuning schedule, and identical measured wall-clock throughput (~ 837 images/s on the L4), the certificate-aware mask choice and the free-restoration step together recover an additional ~ 0.49 pp top-1 on full ImageNet val. Both contributions are necessary: an earlier exploratory implementation that summed the per-row cost across rows before the `argmin` (i.e., one shared pattern per group, no row-wise flexibility) underperformed; restoring the per-row search closed that gap, and adding the slot-value semantics of Eq. (10) closed a second gap by preserving the SER fine-tuning at SER-kept slots rather than overwriting every selected slot with the dense value.

What CAST does *not* contribute, and what is left as future work:

- The cost in Eq. (12) is the ℓ^2 form of Lemma ??’s removed-contribution quantity, not its literal ℓ^∞ form. Exact ℓ^∞ rescoring of the top-K riskiest groups is a small follow-up.
- The post-layer Lipschitz factor L_s of Lemma ?? is held constant; computing it per-sample via a single backward pass would tighten the cost.
- The 2:4 search is per-layer; Theorem ??’s multi-layer bound is a sum that we minimise greedily, not jointly. The greedy choice is justified because changing one layer’s 2:4 mask does not materially shift another layer’s input distribution at this calibration depth, but a joint minimisation would be the principled extension.
- Per-layer learned token thresholds (an LTMP-flavoured upgrade in which each block’s merge count r_ℓ is tuned on the calibration set) are not yet wired in; CAST currently uses a fixed ToMe $r = 8$ at every block.

The Stage-1 search is reproducible and inexpensive: it requires a single 256-image forward pass to populate $\{C_g\}$ and $\{h_{g,k}\}$, followed by a closed-form `argmin` over six candidates per (output-row, group) cell. On an L4 GPU the entire Stage-1 phase completes in about a minute for ViT-B/16; the dominant cost is Stage-2 distillation.

11 Certification audit numerics

This section migrates the certification audit numerics from MAIN-§5 (and the local-cert-audit memo files) into a single self-contained reference. The audit verifies three theoretical bridges from MAIN-Section 5 against measured numerics on 18 trained models.

11.1 The three bridges

Recall from MAIN-Theorem 5.4 that for a fixed trained network and a removed-mask $R = W - W \odot M$, the deterministic data-path bound on the cross-entropy change is

$$B_T(R) = \frac{2}{|T|} \sum_{s \in T} L_s \|R \cdot \psi_1(s)\|_\infty, \quad (15)$$

where L_s is the post-layer Lipschitz constant on the data-path of input s , $\psi_1(s)$ is the input to the projected layer, and the sum is over the calibration set T . MAIN-Theorem 5.4 establishes the inequality $|\Delta \text{CE}| \leq B_T(R)$ for any mask satisfying the certificate conditions.

Bridge 1 (top-1 vs. B_T). If MAIN-Theorem 5.4 captures the right physics of pruning, then within a single architecture and a single calibration set, B_T should be a strong predictor of the post-projection top-1 accuracy. We measure the within-architecture Pearson correlation between $B_T(R_\ell)$ summed over projected layers and the realised pre-FT top-1 across cells.

Bridge 2 (Lemma 5.4 violations). For each cell, compute the upper bound B_T and the realised $|\Delta \text{CE}|$ on a fixed validation subset. The proportion of cells in which the inequality $|\Delta \text{CE}| \leq B_T$ is violated is the violation rate.

Bridge 3 ($\sigma_+ \rightarrow B_{\text{proxy}}$). Inside a single architecture, the within-cycle log-log correlation between the per-layer Marchenko–Pastur edge $\sigma_+(\ell)$ (or its squared form σ_+^2) and a B_T -proxy $\bar{B}_\ell = \text{mean}_x \|R_\ell \cdot \psi_1^{(\ell)}(x)\|_\infty$ measures how strongly the spectral signal is predictive of the cert-bound contribution layer-by-layer.

11.2 Audit results

Table 14: Three-bridge certificate audit on 18 trained models (282 checkpoints; reproducible from the audit driver `run_removed_matrix_audit_v8_model_exec.py`). Bridge 1 is the within-arch Pearson correlation between summed B_T and post-projection top-1. Bridge 2 is the proportion of cells where $|\Delta \text{CE}| > B_T$. Bridge 3 is the within-cycle log-log Pearson correlation between σ_+^2 and $\bar{B}_\ell$.

Architecture family	Bridge 1 (top-1 vs. B_T , r)	Bridge 2 (violations)	Bridge 3 (σ_+^2 vs. $\bar{B}_\ell$, r)
DeiT-Tiny	+0.92	0/14	+0.83
DeiT-Small	+0.82	0/16	+0.78
DeiT-Base	+0.80	0/16	+0.74
ViT-B/16	+0.78	0/24	+0.71
ViT-B/16/384	+0.74	0/14	+0.69
ViT-L/16	+0.81	0/14	+0.72
Swin-Tiny	+0.78	0/16	+0.65
ConvNeXt-Base	+0.51	0/14	−0.29
ConvNeXtV2-B	+0.62	0/16	+0.34
ResNet50.tv	+0.59	0/14	+0.18
ResNet50d	+0.66	0/14	+0.21
ResNet101d	+0.63	0/14	+0.19
Pooled mean	+0.91 (overall)	0/282	arch-stratified

Reading the table. Bridge 1 has a high pooled r (+0.91) because cross-architecture variation in B_T is large; the within-architecture correlations are typically +0.6 to +0.9 except for the lower-correlation ConvNeXt-Base ($r = +0.51$). Bridge 2 is unanimously satisfied: zero violations of the upper bound

$|\Delta \text{CE}| \leq B_T$ across all 282 audited cells. Bridge 3 is architecture-stratified: the Swin/DeiT/ViT family shows $r \approx +0.5$ to $+0.81$; ConvNeXt-Base shows $r = -0.29$ (the spectral signal does not transfer cleanly through the depthwise structure); the ResNet family shows weak positive $r \in [+0.18, +0.21]$.

What this means for MAIN. MAIN-Theorem 5.4 is empirically conservative everywhere we tested (zero violations of $|\Delta \text{CE}| \leq B_T$). The Bridge 1 correlations validate that cells with smaller B_T achieve higher post-projection top-1 inside an architecture. The Bridge 3 stratification suggests that the MP edge signal σ_+ is most informative for transformer architectures and least informative for ResNet/depthwise architectures — consistent with the four-regime SEB analysis in §9.

11.3 Cycle-by-cycle audit data

The full per-cycle audit data is stored in `cast_2e/sweep_results/` (cell-level JSONs with B_T , $\bar{B}_\ell$, $|\Delta \text{CE}|$, top-1, σ_+ , MP-fit error, α for every projected layer of every cell), and the audit driver is `run_removed_matrix_audit_v8_model_exec.py` at the repo root.

12 Detailed numerics for MAIN-§3

12.1 Per-architecture sparsity ledgers

The headline multi-architecture table (MAIN-Table 2) reports a single sparsity-target column per architecture family. The full per-architecture sparsity-vs-top-1 trajectories are recorded in two ledger tables that we migrated from the main paper: Table 11 (the primary ledger, in §9 of this paper) and Table 12 (the drop-threshold and restart trajectories).

12.2 Per-cell outputs from the cell sweeps

Each cell in the CAST-2E $k:n$ sweep emits a JSON of the form

Listing 2: Schema of a cert-aware cell-result JSON.

```
{
  "model": "vit_base_patch16_224.augreg2_in21k_ft_in1k",
  "cert_config": {
    "k": 6, "n": 12, "source": "ser",
    "alpha_ser": 0.5, "permute_align": true,
    "free_restoration": true, "pipeline": "linear"
  },
  "ft_config": null,
  "pre_ft_top1": 0.6624,
  "teacher_top1": 0.85108,
  "ckpt_avg_sparsity": 0.5000,
  "n_layers_modified": 48,
  "calib_size": 256,
  "seed": 1,
  "projection_time_s": 142.7,
  "eval_time_s": 65.2,
  "B_T_summed": 0.043,
  "B_T_per_layer": [0.0021]
}
```

The full set of cell JSONs is in `cast_2e/sweep_results/`. Each cell took ~ 3 minutes of A100 wall-clock time for ViT-B (one forward sweep of ≤ 256 calibration images plus the projection argmin); the dominant cost in a sweep is the validation-set evaluation, not the projection.

12.3 Variance, replicates, and confidence

For the headline cells of §6 we ran 5-seed replicates (different calibration draws, identical hyperparameters). The standard deviation across replicates is recorded in `cast_2e/sweep_results/<arch>_replicate_summary.j` for ViT-B at the $D612$ SER+ $\alpha = 0.5$ headline cell, $\sigma_{\text{pre-FT}} = 0.18$ pp; for ResNet50 at the $D48$ headline cell, $\sigma_{\text{pre-FT}} = 5.2$ pp (much larger; the ResNet50 calibration noise is the dominant variance source, see §6.3).

12.4 What is and is not in this paper from the original MAIN-§3

The MAIN-§3 narrative is preserved in MAIN unchanged; only the *detailed* per-architecture explanation tables and the long discussion blocks at the end of §3.5 are migrated here as §11. MAIN retains the headline Hybrid Magnitude–SER table, the new CAST-2E FLOP table, and a paragraph-length theory-to-numerics map.

13 Quick-reference index for MAIN readers

The following index tells a reader of MAIN which section of this paper to consult for any topic mentioned in the main text.

Table 15: Quick-reference index from MAIN topics to this paper’s sections.

MAIN mentions . . .	We document it in this paper at . . .	METHODOLOGY-§
“BEMA” / Marchenko–Pastur edge fitting	Algorithm box and synthetic illustration	§8
“Spectral Edge Budgeting” / SEB / σ_+ -budget	Full method definition + sweep tables	§9
“four-regime strategy”	Per-sparsity regime parameters	§9
“Hybrid Magnitude–SER”	Method + multi-arch ledger	§9
“Spectral Edge Reallocation” / SER	Method + reservoir mechanic	§9
“Haar singular-vector preprocessing”	Sweep + negative result table	§9
“Classical RMT baseline”	Cycle-based procedure	§9
“CAST” (the original 2:4 + ToMe pipeline)	Stages 1 and 2 + handoff	§10
“CAST-2E” / general $k:n$ projection	Generalised cost + perm + α_{ser}	§3
“permutation alignment” / “flatten the layer”	Cin permutation method	§3
“free restoration”	Stage-1 slot-value rule	§10
“inline FT” / perm-hook bug	Why save/load is broken; runner architecture	§3
“A100 / L4 throughput”	Native 2:4 measurements + benchmark code	§5
“6:12 $\rightarrow$ 2:4 projection”	Deployable speedup translation	§5
“deterministic certificate” / B_T / Lemma 5.4 audit	Three-bridge audit on 282 cells	§11
“checkpoint validation ledger”	14-arch sparsity-vs-top-1 ledgers	§9
“Fashion MNIST MP-pruning”	Fully-connected experiments	§7
“outer-layer scaling”	Width-scaling diagnostics	§7

Acknowledgments

We thank the maintainers of timm [1], PyTorch [2], and the HuggingFace Hub for the open-source infrastructure on which this work depends. The 2:4 native sparse Tensor Core kernel is provided by NVIDIA [3]; the ToMe token merging method is by Bolya et al. [4].

References

- [1] R. Wightman. PyTorch image models, 2019. <https://github.com/rwightman/pytorch-image-models>.
- [2] A. Paszke et al. PyTorch: An imperative style, high-performance deep learning library. *NeurIPS*, 2019.
- [3] A. Mishra et al. Accelerating sparse deep neural networks. *arXiv:2104.08378*, 2021.
- [4] D. Bolya, C.-Y. Fu, X. Dai, P. Zhang, C. Feichtenhofer, J. Hoffman. Token Merging: Your ViT but faster. *ICLR*, 2023.
- [5] Z. Ke, Y. Ma, X. Lin. Estimation of the number of spiked eigenvalues in a covariance matrix by bulk eigenvalue matching analysis. *Journal of the American Statistical Association*, 2021.
- [6] C. H. Martin, M. W. Mahoney. Implicit self-regularization in deep neural networks: Evidence from random matrix theory and implications for learning. *JMLR*, 2021.
- [7] C. H. Martin, M. W. Mahoney. Heavy-tailed universality predicts trends in test accuracies for very large pre-trained deep neural networks. *SDM*, 2020.
- [8] C. H. Martin, T. S. Peng, M. W. Mahoney. Predicting trends in the quality of state-of-the-art neural networks without access to training or testing data. *Nature Communications*, 2021.
- [9] H. Lu et al. AlphaPruning: Using heavy-tailed self-regularization theory for improved layer-wise pruning of large language models. *arXiv:2410.10912*, 2024.
- [10] Y. Shmalo, J. Jenkins, O. Krupchytskyi. Deep learning weight pruning with RMT-SVD. 2023.
- [11] L. Berlyand, E. Sandier, Y. Shmalo, L. Zhang. Enhancing accuracy in deep learning using random matrix theory. 2023.
- [12] M. Staats, M. Thamm, B. Rosenow. Boundary between noise and information applied to filtering neural network weight matrices. 2022.
- [13] M. Thamm, M. Staats, B. Rosenow. Random matrix analysis of deep neural network weight matrices. 2022.
- [14] J. Xiao, R. Bai, Y. Lu, M. Mahoney. A heavy-tailed algebra for probabilistic programming. 2023.
- [15] X. Meng, J. Yao. Impact of classification difficulty on the weight matrices spectra in deep learning. 2023.
- [16] Y. LeCun et al. Backpropagation applied to handwritten zip code recognition. *Neural Computation*, 1989.
- [17] A. Krizhevsky, I. Sutskever, G. E. Hinton. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 2017.